TSTUDIO: Efficient Discovery of Semantic Tokens for Long Time Series Transformers

Rundong Zuo*, Rui Cao*, Guozhong Li†, Byron Choi*, and Sourav S. Bhowmick‡
*Department of Computer Science, Hong Kong Baptist University, Hong Kong SAR, China
†CEMSE Division, King Abdullah University of Science and Technology, Thuwal, Saudi Arabia
‡School of Computing Engineering, Nanyang Technological University, Singapore
*{csrdzuo,csrcao,bchoi}@comp.hkbu.edu.hk, †guozhong.li@kaust.edu.sa, ‡assourav@ntu.edu.sg

***Abstract*—Transformer models for time series have recently demonstrated promising performance. Among others, a semantic tokenization for time series transformers that extracts meaningful tokens from the time series is still in its infancy. In particular, most time series transformers directly take each raw value as a token (a.k.a a value-based token), which does not capture the rich semantics present in time series subsequences. Furthermore, taking numerous value tokens as inputs suffers from the well-known quadratic training-time complexity of transformers. To address this, we propose TSketches, which serve as *semantic tokens* to improve the efficiency and effectiveness of time series transformers. To discover TSketches, we propose a novel framework, called TSTUDIO, that comprises two main components to extract TSketches: (1) a stratified representation learning framework that combines SAX-LSH with a novel STRATified-negAtives triplet loss (STRATALOSS) to learn prototypes of TSketches, serving as a vocabulary of a time series dataset; and (2) a TSKETCHER that discovers TSketches from an input time series based on the prototypes. Extensive experiments on eight longest UCR datasets, and five representative transformers, verify the effectiveness of TSketches for the classification task, where TSketches lead to 2–120× training speed-ups while consistently achieving higher accuracy. In addition, we present a case study to illustrate a healthcare application of TSketches.**

***Index Terms*—Time series classification, efficient transformers, semantic tokenization, subsequence discovery**

I. INTRODUCTION

Time series data arise in diverse domains such as healthcare, smart cities, finance, and economics [1]–[3]. Among the numerous time series tasks, classification stands out as a foundational task that has attracted extensive research [4]–[6].

Transformer-based methods have achieved success in NLP by effectively modeling long-range dependencies with self-attention mechanisms [7]–[9]. A central reason for this success is the tokenization, which decomposes text into discrete tokens and maps them to dense embeddings. Tokens are typically *words/subwords*, as they carry richer semantics than individual *characters*. For example, character-level models (*e.g.*, CANINE [10]) process raw Unicode characters as input (Figure 1(a)), while word-level models employ a BPE tokenizer [11], [12] (Figure 1(b)). This approach not only provides richer semantic information but also reduces the input length for transformers.

In parallel to the success of transformers, researchers have applied them to time series with promising results [17]–[19]. Transformer-based models have achieved strong performance across a wide range of time series tasks, including classification [17], [19], forecasting [20], [21], and anomaly detection [22], demonstrating their ability to model complex temporal dependencies. However, when applied to long time series classification, transformer-based methods still suffer from the following scalability challenges caused by the large number of timestamps in each input sequence.

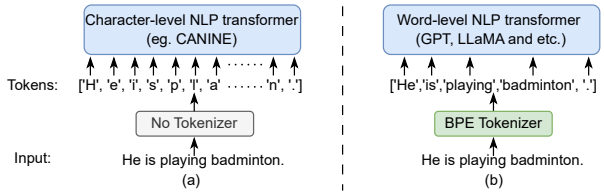


Fig. 1. (a) NLP transformer without tokenizer. (b) NLP transformer with tokenizer, *e.g.*, GPT, LLaMA, and others [13]–[16].

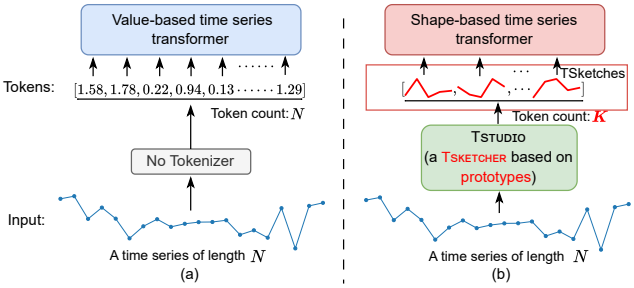


Fig. 2. (a) Value-based time series transformer, *e.g.*, [17]. (b) Time series transformer with TSketches obtained from TSTUDIO.

Challenges: We identify two key challenges in existing time series transformers. (**I**). Most models, referred to as *value-based transformers* [17], [23], directly input individual raw time series values into the transformer (Figure 2(a)). Since transformers have the well-known quadratic computational complexity of the attention mechanism [7], representing each time point as a token makes training inefficient and memory-intensive for long sequences. Furthermore, unlike word tokens in NLP, raw time series values are at the lowest level and lack semantic meaning. (**II**). Recent approaches, termed *shape-based transformers* [19], [24], [25], build tokens from subsequences or shapelets. Although such tokens capture part of the semantics, they still remain two limitations: (i) the discovered shapes often capture only local patterns [26] and are not designed to form semantic vocabulary across time series data, and (ii) shape discovery is computationally expensive [27].

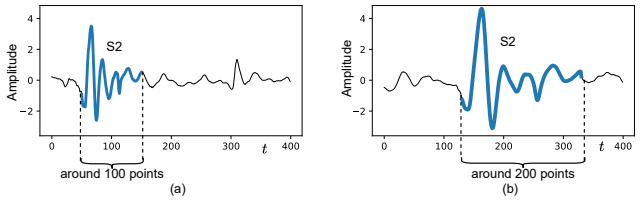


Fig. 3. The *globe scaling* of time series. S2 is the *second heart sound* pattern of *heartbeat sound*. Due to variations in heart rate, S2 (blue) of the two time series differ in their temporal or amplitude scales.

Our approach: We identify that the main reason for the above challenges is the absence of an effective tokenization mechanism for long time series. To fill in the gap, we propose TSketches, a new time series primitive serving as semantic tokens for transformer input (Figure 2(b)), extracted from raw series by our TSTUDIO. Specifically, TSTUDIO first learns a set of prototypes to form a reusable time series vocabulary. Then, for a given input series, it efficiently searches the approximate best-matching subsequence of each prototype as TSketch. Thus, a long time series of length N is represented by K semantic tokens, namely TSketches, with $K < N$, reducing sequence length while improving efficiency and scalability.

The details of TSTUDIO are illustrated in Figure 4. In the offline phase, TSTUDIO learns a set of semantically meaningful prototypes from subsequence collections. We first leverage SAX, which is a well-received foundation of time-series search, and LSH to map subsequences into discrete buckets, and propose the SAX bucket distance with a probabilistic order-preserving (POP) guarantee. The bucket distances can reflect semantic similarity, which allows us to define a stratification of subsequences. On top of this stratification, we propose a novel STRATified-negAtives triplet loss (STRATALOSS). Subsequences are not labeled just by either positives or negatives but at distinguished levels based on their bucket distances to the anchor. To further handle *global scaling* [28], where semantically similar shapes occur at different temporal or amplitude scales (shown in Figure 3), we apply a kernel from a predefined set $\mathcal{H}$ during training. Together, these mechanisms yield representations that are semantically rich subsequences, whose centers form prototypes. Subsequently, the prototypes serve as a vocabulary for time series.

Next, for each prototype, we identify its approximate best-matching subsequence in the input time series, called a TSketch. One may apply a sliding window to obtain subsequences, and match all subsequences for every prototype, which is computationally expensive ($O(NK)$). Moreover, conventional similarity search methods [29]–[32] assume pre-indexed subsequences for ad-hoc searches, which are inapplicable here since subsequences are generated on-the-fly from each input series. Therefore, we propose TSKETCHER, an index built on prototypes for efficient searches of TSketches with complexity reduced to $O(N \log K)$. Each subsequence is routed to its nearest prototype. In particular, we search the subsequence around the prototype and use the best match as the TSketch. As a result, as summarized in Table I, TSketch tokens not only preserve richer semantics but also, improve the training efficiency compared to existing value- or other shape-based tokens.

TABLE I
COMPARISON OF TRAINING AND SEARCH COMPLEXITY ACROSS VALUE-BASED, SHAPE-BASED, AND SKETCH-BASED TRANSFORMERS.

Property	**Value-based**	**Shape-based**	**TSketches**
Tokens' semantic richness	Low	Variable	High
Training complexity	$O(N^2)$	$O(K^2)$	$O(K^2)$
Search complexity	N/A	$O(NK)$	$O(N \log K)$

We conduct extensive experiments on the eight longest time series datasets (length larger than 4000) from the UCR archives [33]. Empirical results demonstrate that using TSketches as transformers' inputs accelerates the training time of a vanilla transformer by 7-120× and other state-of-the-art efficient transformers by 2-40×. In terms of accuracy, TSketch-based models achieve superior or highly competitive performance on most datasets.

Our main contributions are summarized as follows:

- We propose TSTUDIO, a framework that transforms long time series into TSketches. By using K TSketches instead of N values from a long time series to train transformer, the time complexity is reduced from $O(N^2)$ to $O(K^2)$.
- We propose a stratified representation learning through a novel STRATALOSS. The STRATALOSS utilizes the SAX data structure as LSH to stratify negatives to multiple levels, and handles the *global scaling* by applying a kernel to anchor, thus learning a set of semantically-rich prototypes of TSketches as vocabulary for time series.
- We propose TSKETCHER, which efficiently searches for the approximate matching subsequence for each prototype as a TSketch from an input time series. The time series is then represented by K TSketches. The search time complexity is $O(N \log K)$.
- The experiment results show that TSketches achieve clear speedups and accuracy gains. Furthermore, we present a case study using the BinaryHeartbeat dataset. The TSketches we obtained naturally correspond to the "*lub*", "*dub*" and "*systole*", which have been corroborated by medical knowledge.

II. BACKGROUND

In this section, we introduce the key terminologies and notations. Table II summarizes some frequently used symbols.

Time series instance. A time series instance (or simply a *time series*) is denoted as $X = (x_1, x_2, \cdots, x_N)$, where N is the number of timestamps/observations. A time series dataset $\mathcal{D}$ contains a collection of time series instances $|\mathcal{D}|$.

Time Series Subsequences. Given a time series $X = (x_1, x_2, \cdots, x_N)$, a subsequence $T[l:r]$ is defined as the contiguous segment $(x_l,\ x_{l+1},\ \cdots,\ x_r)$, where $1 \le l \le r \le N$. Here, l and r denote the start and end timestamps of the subsequence, respectively.

A. Self-attention mechanism of transformers

The core part of the transformer model is the self-attention mechanism, which captures long-range dependencies by mod-

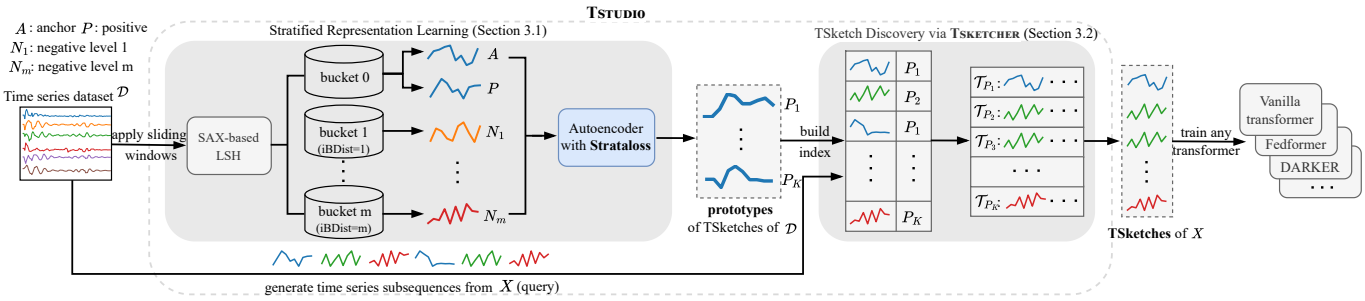


Fig. 4. The overview of our TSTUDIO.

eling pairwise interactions among all input tokens. This process can be described as a function that maps a query and a set of key-value pairs to an output. The self-attention mechanism in the vanilla transformer is formalized as:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \tag{1}$$

where Q, K, and $V \in \mathbb{R}^{N\times d}$ represent the query, key, and value matrices, respectively, for a sequence of N input tokens of dimension d. This formulation computes pairwise similarity scores between all queries and keys, applies softmax normalization, and uses the resulting attention weights to combine the value vectors. While this mechanism enables the model to capture dependencies across arbitrary distances in the sequence, it requires $O(N^2)$ time complexity, making it the major computational bottleneck [34], [35].

We now describe the derivation of the Q, K, and V matrices from input tokens. Given an input sequence $X = \{x_1, x_2, \ldots, x_N\} \in \mathbb{R}^{N\times d}$, where each $x_i \in \mathbb{R}^d$ is a input token. Then:

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v, \tag{2}$$

where W_q, W_k, and $W_v \in \mathbb{R}^{d\times d}$ are learnable weights. For the value-based transformer (Figure 1), each data point is treated as a token, so a series of length N produces N tokens. Since transformer complexity scales quadratically with the number of tokens, training becomes prohibitive for long sequences.

B. Triplet Loss in Time Series

Triplet loss [36] is a recent contrastive objective, defined over triplets of an anchor a, a positive p (similar to a), and a negative n (dissimilar to a). The loss enforces a margin constraint to separate positives from negatives:

$$D_{AP} + \alpha < D_{AN} \tag{3}$$

where $D(\cdot,\cdot)$ is a distance metric (*e.g.*, Euclidean distance) and $\alpha > 0$ is a predefined margin. Minimizing this loss encourages the model to learn an embedding space where similar samples form compact clusters, while dissimilar ones are separated.

ShapeNet [27] extends the triplet framework with a cluster-wise triplet loss, where positives and negatives are defined by k-means clustering, namely positives (the same cluster) and negatives (different clusters). It further introduces an intra-cluster regularization term D_{neg} to constrain negative samples:

$$D_{neg} = \max_{i,j\in(1,K^-)\wedge i<j}\left\{\|f(x_i) - f(x_j)\|_2^2\right\} \tag{4}$$

TABLE II
FREQUENTLY USED NOTATIONS

Notation	Description
X	a time series instance, consists of N time points
T	a time series subsequence generated from X
S_X	TSketches of a time series X
K	the number of TSketch s in S_X
$\mathcal{P}$	the set of prototypes
P	a prototype in $\mathcal{P}$
H	a CNN kernel with fixed weights
$\mathcal{H}$	the set of kernels

where x denotes negative samples and K^- is the number of negatives. This term minimizes the maximum pairwise distance among the negatives, encouraging them to stay close together. These approaches still suffer from a key limitation. Equation 3 applies a fixed margin to all negatives, while Equation 4 penalizes large pairwise distances among negatives. As a result, negatives that are originally far apart in the raw space may collapse into nearby regions in the embedding space in an epoch of the training. The fundamental issue is that they treat all negatives indiscriminately, ignoring their inherent dissimilarity structure. In contrast, our proposed STRATALOSS explicitly stratifies negatives into multiple levels, allowing level-specific margins to preserve these differences.

Problem statement. Given a time series $X = (x_1, x_2, \ldots, x_N)$, the goal is to extract a set of representative subsequences $S_X = \{s_1, s_2, \ldots, s_K\}$, termed TSketches, which serve as input tokens to a transformer model. □

III. TIME SERIES STUDIO (TSTUDIO)

In this section, we propose TSTUDIO to extract time series TSketches. As shown in Figure 4, the framework consists of two main modules: (1) a stratified representation learning to obtain prototypes of TSketches through STRATALOSS; (2) a TSKETCHER to efficiently search the approximate best match TSketch for each prototype.

A. Stratified Representation Learning

To obtain semantically rich prototypes of TSketches, we propose a stratified representation learning framework consisting of two steps: (a) stratification via SAX-LSH and (b) train an autoencoder with STRATALOSS.

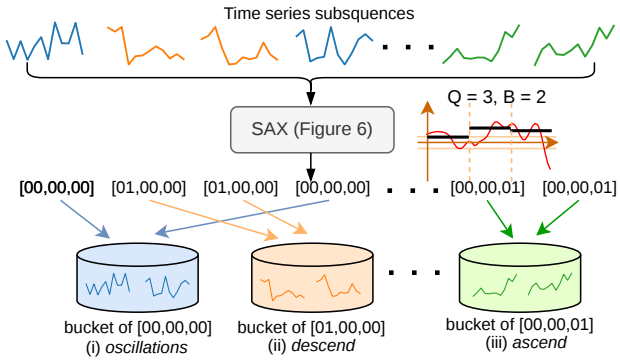


Fig. 5. Illustration of SAX-based Locality-Sensitive Hashing (SAX-LSH). Similar subsequences (in the same color) are hashed to one bucket, yet share the same semantics signified by their shapes (*e.g.*, subsequences in blue are (i) *"oscillations"*, orange are (ii) *"descend"*, and green are (iii) *"ascend"*).

1) Stratification via SAX-LSH.: A central challenge in contrastive learning in time series is how to stratify negative samples for an anchor subsequence. Unlike text, which naturally provides discrete semantic units (*e.g.*, words), subsequences are unlabeled numerical fragments, making it difficult to assign them to meaningful similarity levels.

To address this, we propose a stratification process via SAX-LSH to capture the semantics with SAX bucket distance, which has the Probabilistic Order-Preserving (POP) , as we prove in Theorem 1. In particular, we adopt a well-known foundation for time-series similarity search, namely the symbolic aggregate approximation (SAX) [30], [37] that segments a subsequence into Q parts. We quantize each part into a B-bit code. This yields an initial symbolic word that captures the semantics of the subsequence signified by its shape. We then apply locality-sensitive hashing (LSH) on these codes to group subsequences into buckets: those mapped to the same bucket are likely to share similar semantics, while the distance between buckets provides a principled way to define multiple levels of semantic dissimilarity.

We then present some formal details of this process. Given a time series subsequence $T_i = \{x_l, x_{l+1}, \ldots, x_r\}$, the SAX hash value is defined as:

$$(b_i^1, b_i^2, \ldots, b_i^Q) = h_{\text{SAX}}(T_i), \tag{5}$$

where Q is the number of equal-length segments, and $b_i^Q \in \{0,1\}^B$ is the B-bit representation for each segment.

The resulting hash values have a length of $Q * B$ bits, and subsequences sharing the same hash value are assigned to the same bucket. Figure 5 illustrates the process: each subsequence is mapped to an SAX hash value, and subsequences with very similar SAX hash values are routed to the same bucket, implying similar semantic information. For example, we observe that the subsequences in blue, orange, and green are*oscillation*, *descend*, and *ascend*, respectively.

Based on the SAX hash value, we further define a distance metric, *SAX bucket distance*, to measure the distance between buckets, as presented in Definition 1:

Definition 1. ***(SAX bucket distance)*** *Given two time series subsequences $T = (t_1, \ldots, t_n), U = (u_1, \ldots, u_n)$, and their corresponding SAX hash values $b_T = (b_T^1, \ldots, b_T^w), b_U = (b_U^1, \ldots, b_U^w)$, where each segment b_T^k, b_U^k is a B-bit binary code, the distance between the k-th segments is defined as follows:*

$$\text{dist}_{iB}(b_T^k, b_U^k) = |\textstyle\sum_{i=0}^{B-1} 2^i \cdot (a_i - c_i)|, \tag{6}$$

where $b_T^k = a_{B-1} \ldots a_0$ and $b_U^k = c_{B-1} \ldots c_0$.

The SAX bucket distance between b_T and b_U is defined as:

$$\text{iBDist}(b_T, b_U) \equiv \textstyle\sum_{k=1}^{w} dist(b_T^k, b_U^k) \tag{7}$$

It is worth noting that, unlike traditional SAX-based measures such as MinDist [38], which are designed to lower-bound the Euclidean distance and thus preserve absolute distances, our SAX bucket distance possesses a key property called *Probabilistic Order-Preserving (POP)* focusing on relative similarity ordering among subsequences. Specifically, given two pairs of subsequences, if $iBDist(b_{T_i}, b_{T_j}) < iBDist(b_{T_k}, b_{T_l})$, then with high probability (w.h.p.) their true Euclidean distances satisfy $Dist(T_i, T_j) < Dist(T_k, T_l)$. This property enables us to stratify negatives into multiple levels in representation learning, where the distance between buckets reflects the increase in dissimilarity, laying the foundation for learning discriminative and semantically rich prototypes. We then formally state the POP property as the following Theorem 1:

Theorem 1 (Probabilistic Order-Preserving Property of SAX bucket distance). *Let $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ denote the normal distribution with mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\Sigma}$. Let w be the number of (Piecewise Aggregate Approximation (PAA)) segments, and let $T = (t_1, \ldots, t_n)$, $U = (u_1, \ldots, u_n)$ be two time series subsequences with $t_1, \ldots, t_n, u_1, \ldots, u_n \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1)$. Assume their SAX bucket distance is $iBDist(b_T, b_U) = d_{iB}$. Then, for any $\epsilon, \delta > 0$, the following inequality holds:*

$$\begin{aligned}\Pr\Big[\|T-U\|_2^2 \le n\delta^2 + d_{iB}\tfrac{n}{w}(d_z^2 + 2\delta d_z) + 2(n-w) + \epsilon\Big] \\ \ge \big(2\Phi(\sqrt{\tfrac{n}{2w}}(d_z+\delta)) - 1\big)^{d_{iB}} \cdot \big(2\Phi(\sqrt{\tfrac{n}{2w}}\max(\delta, d_z)) - 1\big)^{w-d_{iB}} \\ \cdot \frac{\epsilon^2}{8n(1-\frac{w}{n})^2 + \epsilon^2},\end{aligned} \tag{8}$$

where $d_z = \Phi^{-1}(1 - \frac{1}{2^B}) - \Phi^{-1}(1 - \frac{2}{2^B})$ is a constant, $\Phi(\cdot)$ is CDF of the standard Gaussian distribution and B is the number of SAX bits per segment.

Analysis of Theorem 1. Here, we summarize key equations to illustrate the intuitions behind Theorem 1 of SAX bucket distance, and the detailed proof is provided in Appendix A. Intuitively, the probabilistic bound is determined by the term $(2\Phi(\sqrt{\frac{n}{2w}}(d_z+\delta))-1)^{d_{iB}} \cdot (2\Phi(\sqrt{\frac{n}{2w}}\max(\delta, d_z)-1))^{w-d_{iB}}$, while $\frac{\epsilon^2}{8n(1-\frac{w}{n})^2+\epsilon^2}$ can be close to one since ϵ is an arbitrary positive value. Thus, the final probability is dominated by d_{iB} and δ in the former term since n, w and d_z are given. Furthermore, a smaller d_{iB} leads to a higher probability of yielding a tighter upper bound on the Euclidean distance between T and U. Thus, subsequences with similar SAX hash values can achieve a high probability of lying within a small-radius sphere in the original space.

By contrast, the MinDist metric [38] in traditional SAX is

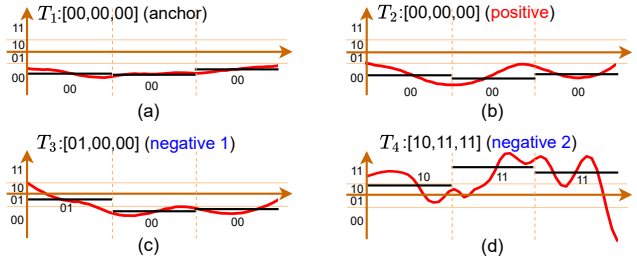


Fig. 6. Examples of SAX values. Subsequences are converted to symbols/bits by applying PAA and then discretizing the results.

TABLE III
THE DISTANCES BETWEEN T_1 AND THE OTHERS

Pair	Euclidean	MinDist	SAX bucket dist.
(T_1, T_2)	2.06	0	0
(T_1, T_3)	4.03	0	1
(T_1, T_4)	12.94	8.21	8

designed as a lower bound to facilitate fast retrieval [30], but it offers weaker probabilistic guarantees of relative similarity order under normal distribution assumptions on input time series subsequences. A key limitation lies in its treatment of adjacent intervals: MinDist assigns a value of zero when two PAA segments fall into neighboring intervals, whereas our SAX bucket distance assigns a value of one. Consequently, MinDist cannot distinguish subsequences in adjacent symbolic intervals from those with identical symbols.

To further illustrate the computation of SAX bucket distance and its POP property in Theorem 1, we provide the Example 1 with four subsequences compared to MinDist.

Example 1. *Consider four time series subsequences T_1, T_2, T_3, and T_4, along with their corresponding SAX values as illustrated in Figure 6. Thus, T_1 and T_2 are assigned to the same bucket $[00, 00, 00]$, T_3 to $[01, 00, 00]$, and T_4 to $[10, 11, 11]$. The distances between T_1 and the others are presented in Table III.*

Example 1 provides three key insights. First, SAX-LSH correctly groups the most similar pair T_1 and T_2 into the same bucket. Second, the SAX bucket distance preserves the relative ordering of Euclidean distances (Theorem 1), since $dist_{iB}(T_1, T_4) > dist_{iB}(T_1, T_3)$ matches $dist(T_1, T_4) > dist(T_1, T_3)$. Third, unlike MinDist, our SAX bucket distance can distinguish between T_1 and T_3, further underscoring MinDist's limitations regarding the POP property.

2) Training autoencoder with STRATALOSS*:* With negatives stratified into multiple levels by SAX-LSH, the next step is to learn embeddings that preserve semantic information in a low-dimensional space. To show how we achieve this, we adopt an autoencoder $model_A$ consisting of an encoder $f_e(\cdot)$ and a decoder $f_d(\cdot)$ as model structure and propose a novel STRATALOSS for training.

Assigning multiple margins based on SAX bucket distance. STRATALOSS leverages bucket-based stratification to assign leveled margins between positives and negatives. Specifically, in each training batch, an anchor A and a positive P are sampled from the same SAX-LSH bucket, while all remaining subsequences from other buckets serve as negative candidates. These negatives are stratified into multiple levels based on their SAX bucket distances from the anchor's bucket.

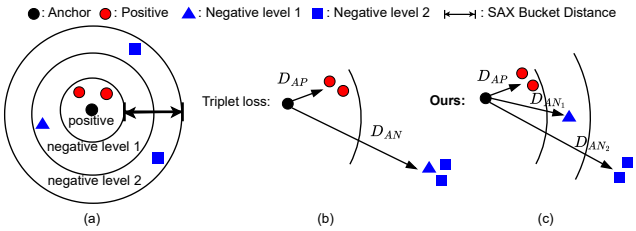


Fig. 7. (a) The negative levels determined by SAX bucket distances. (b) Existing triplet loss [27], [36], [39], setting a fixed margin for all negatives. (c) STRATALOSS, setting multiple margins based on the (a).

Figure 7 contrasts our method with previous triplet-based approaches [27], [36], [39]. In the left-hand side of the figure, the samples are organized into a positive set and multiple negative levels based on SAX bucket distances (different shapes indicate different levels). As shown in Figure 7(b), previous methods enforce a fixed margin [27], which collapses negatives from distinct semantic levels and obscures their true separations. In contrast, STRATALOSS assigns level-specific margins that increase with the SAX distance in Figure 7(c). We remark that when m=1, STRATALOSS reduces to the standard triplet loss.

More formally, for a negative sample N_i at level i, we enforce the following constraint in the embedding space:

$$D_{AP} + \frac{i}{m}\alpha < D_{AN_i}, \tag{9}$$

where D_{AP} and D_{AN_i} denote the distances between the anchor–positive and anchor–negative pairs, respectively, m is the total number of negative levels (with $m = Q*(2^B - 1)$), and α is the maximum margin parameter. This formulation ensures that positives remain closer to the anchor than negatives and that the margin increases with the negative level i. As a result, semantically similar subsequences are mapped closer together in the embedding space. Conversely, dissimilar subsequences are pushed farther apart, such that the separation distance reflects their SAX bucket distance. As a result, by setting multi-level margins to prevent samples from collapsing into a single negative class, STRATALOSS yields a low-dimensional embedding space that is not only highly discriminative, but also preserves the semantic SAX structure.

Based on Equation 9, for a negative sample x_{n_i} (a subsequence) at level i, the corresponding term in loss function is computed by:

$$\mathcal{L}_i = \max\left(d(f_e(x_a), f_e(x_p)) - d(f_e(x_a), f_e(x_{n_i})) + \frac{i}{m}\alpha, 0\right) \tag{10}$$

where $d(\cdot,\cdot)$ denotes the distance, α is the maximum margin, and $f_e(\cdot)$ is the encoder.

Applying kernels for *global scaling*. As we have shown in Figure 3, subsequences that differ in temporal or amplitude scale may still have the same semantic meaning (*i.e.*, the same prototype), which is called *global scaling* in time series. To mitigate the impact of global scaling on prototypes, we

Algorithm 1: Training $model_A$ with STRATALOSS

Input: Time series subsequences $\mathcal{T}$ obtained from $\mathcal{D}$, the SAX buckets $\mathcal{B}$ of each $\mathcal{T}$, the kernel set $\mathcal{H}$
Output: The trained model $model_A$

1 $model_A$.init(); // Initialize $model_A$ consists of a encoder $f_e(\cdot)$ and a decoder $f_d(\cdot)$
2 **for** $epoch \in \{0, \cdots, epochs\}$ **do**
3 // Computing STRATALOSS
4 $x_a, x_p, b_a \leftarrow$ Random select$(\mathcal{T}, \mathcal{B})$;
5 $\{x_{n_1}, x_{n_2}, x_{n_3}, \cdots, x_{n_m}\} \leftarrow$ SelectNegative$(\mathcal{T}, \mathcal{B}, b_a)$;
6 **for** i *in* $range(1, m+1)$ **do**
7 $\mathcal{L}_i \leftarrow \max\left(d(f_e(x_a), f_e(x_p)) - d(f_e(x_a), f_e(x_{n_i})) + \frac{i}{m}\alpha, 0\right)$ // Equation 10
8 $\mathcal{L}_n \mathrel{+}= \mathcal{L}_i$;
9 $H \leftarrow$ Random pick$(\mathcal{H})$;
10 $\mathcal{L}_{sc} \leftarrow d(f_e(x_a), f_e(H * x_a))$ // Equation 11
11 $\mathcal{L}_{Strataloss} \leftarrow \mathcal{L}_{sc} + \frac{1}{m}\mathcal{L}_n$ // Equation 12
12 // Computing reconstruction loss
13 **for** T_i *in* $\mathcal{T}$ **do**
14 $\mathcal{L}_{r_i} = \|T_i - f_d(f_e(T_i))\|^2$ // Equation 14
15 $\mathcal{L}_{re} \mathrel{+}= \mathcal{L}_{r_i}$;
16 $\mathcal{L}_{all} \leftarrow \lambda\mathcal{L}_{triplet} + (1-\lambda)\mathcal{L}_{re}$ // Equation 13
17 $model_A$.backward$(\mathcal{L}_{all})$;
18 **return** $model_A$

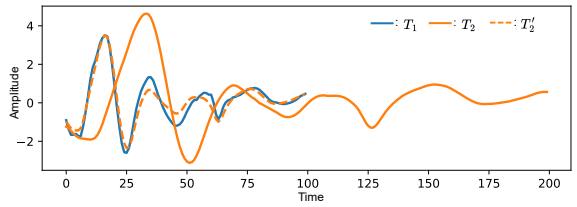


Fig. 8. Application of kernel to tackle *global scaling* of T_1 and T_2, where T_1 and T_2 are *the second heart beat*, $T_2' = H * T_2$, and H is a kernel.

incorporate a scale-consistency term $\mathcal{L}_{sc}$:

$$\mathcal{L}_{sc} = d(f_e(x_a), f_e(x_a')) \tag{11}$$

where $x_a' = H * x_a$ is obtained by applying a convolution kernel H to the anchor. The kernel H is selected from a kernel set $\mathcal{H}$ that contains kernels with various strides. The weights for these kernels are sampled from a Gaussian distribution centered on a mean filter, thus resulting in a time complexity of only $O(|T|)$ for this process.

Figure 8 provides an example of how convolutional kernels address the *global scaling*. T_1 and T_2 are the two subsequences mentioned in Figure 3. After convolving the longer subsequence T_2 with a kernel whose weights are $[0.38, 0.38]$, and a stride of 2, we obtain T_2', which is now much closer in distance to T_1. This reduced distance ensures that the model perceives them as semantically similar, thereby treating them as TSketches of the same prototype during training. $\mathcal{L}_{sc}$ minimizes their embedding distance, thereby enhancing prototype invariance to scale variations. While convolutional kernels address most *global scaling*, we resort to Dynamic Time Warping (DTW) for the case where subsequences still exhibit minor mismatched lengths after the operation.

The final Stratified-Negatives triplet loss is defined as the sum of $\mathcal{L}_{sc}$ and the average triplet loss across all negative levels $\mathcal{L}_i$:

$$\mathcal{L}_{Strataloss} = \mathcal{L}_{sc} + \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}_i \tag{12}$$

To ensure reconstruction fidelity in the learned embedding space of autoencoders, we formulate a composite loss function $\mathcal{L}_{all}$ that linearly combines our proposed $\mathcal{L}_{Strataloss}$ and a reconstruction loss $\mathcal{L}_{re}$:

$$\mathcal{L}_{all} = \lambda\mathcal{L}_{Strataloss} + (1-\lambda)\mathcal{L}_{re} \tag{13}$$

where $\lambda \in [0, 1]$ is a hyperparameter to balance their contributions. $\mathcal{L}_{re}$ is defined as the mean squared error (MSE) between x_i and its reconstruction $\hat{x}_i$ via the autoencoder:

$$\mathcal{L}_{re} = \|x_i - f_d(f_e(x_i))\|^2 \tag{14}$$

where $f_e(\cdot)$ and $f_d(\cdot)$ denote the encoder and decoder.

Algorithm 1 presents the details of stratified representation learning. The input consists of time series subsequences $\mathcal{T}$ generated by sliding windows on $\mathcal{D}$, along with the corresponding SAX bucket for each $T \in \mathcal{T}$. For clarity, we omit the standard mini-batch partitioning and present a simplified training view over the full training set across all epochs. We begin by initializing the autoencoder $model_A$, consisting of an encoder $f_e(\cdot)$ and a decoder $f_d(\cdot)$ (Line 1). For each epoch (Line 2), we randomly sample an anchor x_a, and positives x_p (Line 4). Negative samples are stratified into m levels based on their SAX bucket distance to b_a (Line 5). The stratified triplet loss $\mathcal{L}_n$ is then computed by aggregating the losses across all negative levels (Lines 6–8). Meanwhile, we compute $\mathcal{L}_{sc}$ by randomly picking a kernel H from the kernel set $\mathcal{H}$ (Lines 9-10). The overall STRATALOSS is then calculated as $\mathcal{L}_{sc} + \frac{1}{m}\mathcal{L}_n$ (Line 11). Subsequently, the reconstruction loss $\mathcal{L}_{re}$ is computed across all subsequences $T_i \in \mathcal{T}$ via the autoencoder (Lines 12–15). Finally, $\mathcal{L}_{all}$ is formed as the weighted sum of STRATALOSS and $\mathcal{L}_{re}$ (Line 16), and used to update the model parameters via backpropagation (Line 17).

Prototype Extraction. After training the autoencoder $model_A$, we extract representative prototypes of time series subsequences. First, the encoder $f_e(\cdot)$ projects all subsequences $\mathcal{T}$ into low-dimensional representation space, generating the set $\mathcal{R}$ (Lines 2–5). Next, K-Means clustering is applied to $\mathcal{R}$ to obtain K cluster centers $\mathcal{C} = \{C_1, C_2, \ldots, C_K\}$ (Line 6), which are the latent representations of prototypes. Each cluster center is then decoded back into the original time series space via $f_d(\cdot)$, producing the final prototype set $\mathcal{P} = \{P_1, P_2, \ldots, P_K\}$ in the original time series space (Lines 7–10).

B. *TSketch search via* TSKETCHER

In Section III-A2, we have constructed a set of K prototypes $\mathcal{P}$ for a time series dataset. To efficiently compute the tokens for a long time series, we formulate the problem of TSketch search as follows:

[**TSketch search problem**] Given a prototype set $\mathcal{P} = \{P_1, P_2, \ldots, P_K\}$ obtained from a time series dataset $\mathcal{D}$, for an input time series $X = \{x_1, x_2, \ldots, x_N\}$, the *TSketch search*

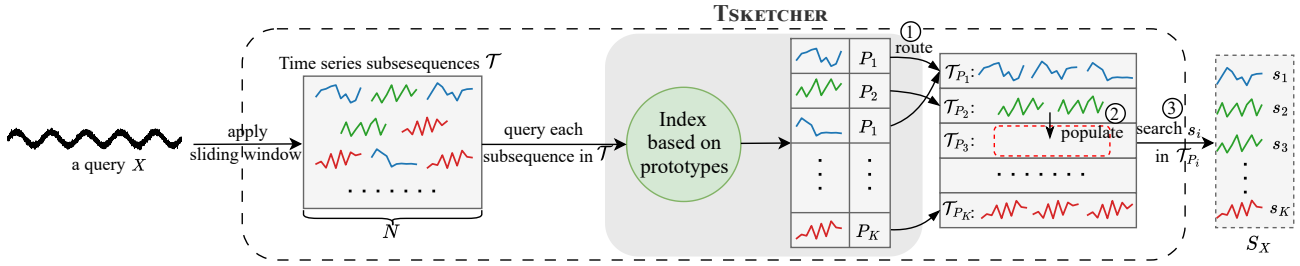


Fig. 9. TSketch search process. Subsequences are mapped to their nearest prototypes with an index. Empty sets are then populated with subsequences the same as the nearest non-empty candidate set. Finally, TSketches are selected as the closest subsequences to each prototype.

Algorithm 2: Prototype Extraction

Input: Time series subsequences $\mathcal{T}$ obtained from $\mathcal{D}$, the trained model $model_A$
Output: prototypes $\mathcal{P}$

Initialize set $\mathcal{R}$, $\mathcal{P}$;
// Computing the representations $\mathcal{R}$ based on $f_e(\cdot)$
for T *in* $\mathcal{T}$ **do**
  $R \leftarrow f_e(T)$;
  $\mathcal{R}$.append(R);
$\mathcal{C} : \{C_1, C_2, ..., C_K\} \leftarrow$ kmeans($\mathcal{R}$);
// Computing the prototypes $\mathcal{P}$ based on $f_d(\cdot)$
for C *in* $\mathcal{C}$ **do**
  $P \leftarrow f_d(C)$;
  $\mathcal{P}$.append(P);
return $\mathcal{P}$

problem is to search a set of TSketches $S_X = \{s_1, s_2, \ldots, s_K\}$ for X, such that

$$s_i = \arg\min_{T_j \in \mathcal{T}} dist(T_j, P_i), \tag{15}$$

where $\mathcal{T}$ is the set of subsequences obtained by applying a sliding window to X.

The primary challenge of the TSketch search problem lies in achieving this with sub-linear complexity relative to the $O(NK)$ brute-force approach in previous works [19], [24], [25]. In addition, this problem cannot be directly addressed by conventional similarity search techniques [28]–[32], as they typically build indexes on a large number of subsequences offline for ad-hoc queries. In contrast, in our problem, the subsequences are generated from the input time series X on-the-fly, which serve as the *queries*, and are determined to be TSketches when they best-match the prototypes (Equation 15). Such a query paradigm is analogous to publish/subscribe systems, where the subsequences are (i) matched against a relatively small number of prototypes, and (ii) routed to the candidate sets of their prototypes for further matching.

Steps of TSKETCHER**.** To solve the TSketch search problem, we propose TSKETCHER, which efficiently computes TSketches that are approximate best-matches of the prototypes with a probabilistic guarantee on recall. The overview of TSKETCHER is shown in Figure 9. We construct a tree-based index on the prototypes offline once. For a concrete presentation, we simply adopt the ball tree (denoted as BallTree). The steps of TSKETCHER are also shown in Algorithm 3. In query time, we take an input time series X, apply a sliding window

Algorithm 3: TSketches search from Prototypes $\mathcal{P}$

Input: a time series X of length N, prototypes $\mathcal{P}$, a $BallTREE$ built on $\mathcal{P}$, $\mathcal{P}_{nearest}$ the nearest neighbors of $\mathcal{P}$
Output: TSketch set S of time series X

1 Initialize TSketch set $S = \emptyset$ for X;
2 Initialize $\{\mathcal{T}_{P_k} = \emptyset\}_{k=1}^{K}$;
3 $\mathcal{T} : \{T_1, T_2, T_3, ..., T_N\} \leftarrow$ slideTS(X);
4 // Take the nearest prototypes to map $\mathcal{T} = \{T_1, T_2, T_3, ..., T_N\}$
5 **for** T *in* $\{T_1, T_2, T_3, ..., T_N\}$ **do**
6 $P_k \leftarrow BallTREE$.query(T);
7 $\mathcal{T}_{P_k}$.append(T)
8 // Search TSketch for each prototype P_i in its candidate set $\mathcal{T}_{P_i}$
9 **for** P_i *in* $\mathcal{P}$ **do**
10 // If P_i is not assigned to any T as its label, take $\mathcal{T}_{P'}$ as $\mathcal{T}_{P_i}$
11 **while** $\mathcal{T}_{P_i} == \emptyset$ **do**
12 // Take the nearest leaf node P' to P
13 $P' \leftarrow \mathcal{P}_{nearest}$.get($P_i$);
14 $\mathcal{T}_{P_i} \leftarrow \mathcal{T}_{P'}$;
15 // Take the nearest subsequence in $\mathcal{T}_{P_i}$ to be a TSketch s_i corresponding to prototype P_i
16 $s_i \leftarrow argmin_{T \in \mathcal{T}_{P_i}}(dist(P_i, T))$;
17 S.append(s_i)
18 **return** S

to obtain a set of time series subsequences $\mathcal{T}$: $\{T_1, \ldots, T_N\}$ (Line 3)[1] ① *Routing*: We query each subsequence T in the index of prototypes for its nearest prototype P_k, and route it to the candidate set $\mathcal{T}_{P_k}$ (Lines 5–7). For example, both blue subsequences have P_1 as their nearest neighbor and they are routed to the same set $\mathcal{T}_{P_1}$. ② *Populating*: Next, there can be the case that the candidate set $\mathcal{T}_{P_i}$ of a P_i is empty. In this case, we fetch P_i's nearest prototype P' that has non-empty $\mathcal{P}_{nearest}$ and set its candidate set to $\mathcal{T}_{P_i}$ (Lines 10–14). For instance, if any prototype's set remains empty ($\mathcal{T}_{P_3}$ in Figure 9), we populate the empty set with the subsequences from the nearest non-empty set ($\mathcal{T}_{P_2}$ in Figure 9). ③ *Searching*: For each prototype P_i, we compute the subsequence in $\mathcal{T}_{P_i}$ that is nearest to P_i as TSketch s_i

[1]One may generate further subsequences of variable lengths and extend the *dist* function to have a more comprehensive search of prototypes. This increases N and hence, the runtime. As a proof of concept, our experiments show that generating N subsequences is sufficient to have a significant improvement in accuracy.

(Lines 15–17). A baseline method to compute s_i for a given P_i is to search within $\mathcal{T}_{P_i}$ and its nearby candidate set, which returns the best matching. We prove that searching within $\mathcal{T}_{P_i}$ alone guarantees a probabilistic lower bound on recall, presented in Theorem 2. Hence, we search s_i in P_i in Line 16. Finally, Algorithm 3 yields a total of K TSketches, denoted as $S_X = \{s_1, s_2, \cdots, s_K\}$. Each TSketch s_i is for the corresponding prototype P_i, which represents X for model training. We give an example to illustrate this process.

Example 2. *Consider two prototypes $P_1 = (0.12, 0.27)$ and $P_2 = (0.89, 0.77)$, along with two subsequences $T_1 = (0.83, 0.72)$ and $T_2 = (0.91, 0.84)$.*
① Routing. Both T_1 and T_2 are closer to P_2 than to P_1, since $dist(P_2, T_1) = 0.08 < 0.84 = dist(P_1, T_1)$ and $dist(P_2, T_2) = 0.07 < 0.97 = dist(P_1, T_2)$. Thus, $\mathcal{T}_{P_1} = \emptyset$ and $\mathcal{T}_{P_2} = \{T_1, T_2\}$.
② Populating. Since $\mathcal{T}_{P_1}$ is empty, $\mathcal{T}_{P_1}$ is populated with subsequences from $\mathcal{T}_{P_2}$, as P_2 is the nearest prototype to P_1. Hence, $\mathcal{T}_{P_1} = \{T_1, T_2\}$.
③ Searching. For P_1, the nearest subsequence in $\mathcal{T}_{P_1}$ is T_1 ($dist(P_1, T_1) = 0.84 < 0.97$). For P_2, the nearest subsequence is T_2 ($dist(P_2, T_2) = 0.07 < 0.08$). The selected TSketches are $s_1 = T_1$ and $s_2 = T_2$.

Theorem 2 (Probabilistic lower bound guarantee of Algorithm 3). *Given time series prototypes $\mathcal{P} = \{P_1, \ldots, P_K\}$ and time series subsequences $\mathcal{T} = \{T_1, \ldots, T_N\}$ where $P_1, \ldots, P_K, T_1, \ldots, T_N \overset{\text{iid}}{\sim} \mathcal{N}(0, I_d)$. I_d denotes the $d \times d$ identity matrix, TSketch s_i returned by Algorithm 3 has the following property:*

$$\Pr\left[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)\right] \geq 1 - \exp\left(-\alpha(d) \cdot \frac{N}{K}\right), \tag{16}$$

where $\mathrm{NN}_A(x)$ denotes the nearest neighbor search (NN) of x in set A and $\alpha(d)$ is a positive constant dependent only on the dimension d.

Proof. According to Algorithm 3, we can rewrite the probability with the expectation below:

$$\Pr[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)] = \mathbb{E}_{\mathcal{T},\ \mathcal{P}}\left[\frac{1}{K}\sum_{i=1}^{K} \mathbf{1}\left(\mathrm{NN}_{\mathcal{P}}\left(\mathrm{NN}_{\mathcal{T}}(P_i)\right) = P_i\right)\right] \tag{17}$$

where $\mathbf{1}$ denote the indicator function. Given $\mathcal{P} = \{P_1, \ldots, P_K\} \subset \mathbb{R}^d$, the Voronoi region of P_i is

$$V(P_i) = \{x \in \mathbb{R}^d | \|x - P_i\| \leq \|x - P_j\|, \forall j \neq i\} \tag{18}$$

For each P_i, we explore its nearest neighbor $T^* = \mathrm{NN}_{\mathcal{T}}(P_i)$. If $T^* \in V(P_i)$, we have $p_i = \mathrm{NN}_{\mathcal{T}}(P_i)$. Thus the probability holds

$$\Pr[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)] = \mathbb{E}_{\mathcal{T},\ \mathcal{P}}\left[\frac{1}{K}\sum_{i=1}^{K} \mathbf{1}\left(T^* \in V(P_i)\right)\right] \tag{19}$$

Assume $R_i = \min_{j \neq i} \frac{1}{2}\|P_i - P_j\|$ representing the minimum radius of the Voronoi region of P_i. If $T^* \in B(P_i, R_i)$, then T^* lies in $V(P_i)$. Thus, we calculate the probability $\Pr[\|T^* - P_i\| < R_i] = \Pr[T^* \in V(P_i)]$.

Let $Z_j = \|T_j - P_i\|$ and $z_j = T_j - P_i \sim \mathcal{N}(0, 2I_d)$. Then we achieve $\|z_j\|^2 \sim 2\chi_d^2$ and $Z_j = \|T_j - P_i\| \sim \sqrt{2}\chi_d$. The distribution of the nearest neighbor distance R_{NN} is shown below:

$$F_{NN}(r) = \Pr[\min_j Z_j < r] = 1 - (1 - F_{\chi_d}(\tfrac{r}{\sqrt{2}}))^N \tag{20}$$

TABLE IV
STATISTICS OF DATASETS USED IN OUR EXPERIMENTS.

Dataset	Train Size	Test Size	Length
RightWhaleCalls	10,934	1,962	4,000
KeplerLightCurves	920	399	4,767
FruitFlies	17,259	17,259	5,000
FaultDetectionA	10,912	2,728	5,120
CatsDogs	138	137	14,773
BinaryHeartbeat	204	205	18,530
UrbanSound	2,713	2,712	44,100
DucksAndGeese	50	50	236,784
Synthetic	1,000	1,000	2^9–2^{17}

Therefore,

$$\Pr[T^* \in V(P_i)] \geq \Pr[R_{NN} < R_i] = 1 - (1 - F_{\chi_d}(\tfrac{R_i}{\sqrt{2}}))^N \tag{21}$$

For $P_i \sim \mathcal{N}(0, I_d)$, we can obtain $\mathbb{E}[R_i] = \mathbb{E}[\frac{1}{2}\min_{j \neq i}\|P_i - P_j\|] = (\frac{2}{K})^{1/d} - (\frac{1}{K})^{1/d} = c_d \cdot K^{-1/d}$ according to the theory on nearest neighbor in [40] where c_d is a constant dependent on d. When $R_i \ll 1$, the Cumulative Distribution Function (CDF) of the χ distribution can be denoted as:

$$\begin{aligned} F_{\chi_d}(\frac{R_i}{\sqrt{2}}) &= \frac{1}{\Gamma(d/2)}\int_0^{R_i^2/4} t^{d/2-1}e^{-t}dt \geq \frac{1}{\Gamma(d/2)}\int_0^{R_i^2/4} t^{d/2-1}dt \\ &= \frac{1}{\Gamma(d/2)} \cdot \frac{(R_i^2/4)^{d/2}}{d/2} = \frac{1}{2^d\Gamma(d/2+1)}R_i^d \end{aligned} \tag{22}$$

Finally, we achieve

$$\begin{aligned} \Pr[R_{NN} < R_i] &\geq 1 - (1 - \frac{1}{2^d\Gamma(d/2+1)}(c_d \cdot K^{-1/d})^d)^N \\ &= 1 - (1 - \alpha(d)K^{-1})^N \geq 1 - \exp(-\alpha(d) \cdot \frac{N}{K}) \end{aligned} \tag{23}$$

and $\alpha(d) = \frac{c_d^d}{2^d\Gamma(d/2+1)}$, *i.e.*, $\Pr[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)] \geq \Pr[R_{NN} < R_i] \geq 1 - \exp(-\alpha(d) \cdot \frac{N}{K})$, where $\alpha(d)$ is a constant dependent to d. □

Analysis of Theorem 2 As observed in Equation 16, the lower bound increases with the ratio of N to K. This shows that TSKETCHER is particularly suitable for our target scenario: extracting TSketches (K) from a large number of subsequences (N) from a long time series X. Furthermore, the equation also has a constant $\alpha(d)$ that penalizes high dimensionality. Thus, we operate TSKETCHER on the low-dimensional embedding space after the encoder in Section III-A2. In addition, we also prove in the supplementary material (Appendix B) that a similar lower-bound guarantee exists when our TSKETCHER operates on data following other distributions that can be approximated by Gaussian mixture models (GMMs).

Time complexity analysis. ① Routing: Building the index on prototypes allows each of the N subsequences to be searched in $O(\log K)$, leading to a total cost of $O(N \log K)$. ② Populating: Finding the empty candidate set to populate it takes $O(K)$ time. ③ Searching: For each prototype P_i, we identify its nearest subsequence s_i from the candidate set $\mathcal{T}_{P_i}$. Since $\sum_{i=1}^{K} |\mathcal{T}_{P_i}| = N$, the complexity of this step requires $O(N)$. Thus, the total time complexity of TSKETCHER is $O(N + K + NlogK) = O(NlogK)$.

IV. Experimental evaluation

We empirically evaluate TSketches for time series transformers, covering the experimental setup (Section IV-A), overall evaluation (Section IV-B), ablation studies on StrataLoss (Section IV-C) and TSketcher (Section IV-D), and a case study (Section IV-F). The appendix and source code are provided as supplemental material at the following link[2].

A. Experiment Setup

Environment. All experiments were executed on a server with a NVIDIA V100-32G GPU and an Intel Xeon Gold 6226 CPU operating at 2.70GHz. The software stack was built upon Python 3.8, with PyTorch 1.10.0.

Datasets. We evaluate TSketch on the eight longest time series datasets from the UCR Time Series Archive [33] since their length is larger than 4000. To further investigate scalability, we also generate synthetic datasets with lengths varying systematically, ranging from 2^9 to 2^{17}. The details of the datasets are provided in Table IV.

Evaluation Metrics. We assess the performance of our proposed method from two perspectives: efficiency and effectiveness. For efficiency, we report the training time of each method. For effectiveness, we used time series classification, as it has comprehensive results for comparison. Our primary metric is accuracy. Following the previous works [19], [24], [27], we provide a robust evaluation by also reporting the average rank across all datasets. We also perform Friedman and Wilcoxon signed-rank tests to compare TSketch with competing baselines.

Evaluation Details. For fair comparison in training time, all baselines were run with identical hyperparameter settings in each dataset. The search of TSketcher of the training set can be considered a preprocessing step of the transformer training. Meanwhile, since its runtime is *negligible* compared to the transformer's training time, we exclude it from the training time measurements and report it separately in Section IV-C. For accuracy evaluation, we perform hyperparameter tuning by splitting the training set into 80% for training and 20% for validation, where the validation set was used to optimize transformer-specific hyperparameters. The validation set is used specifically to optimize transformer-related hyperparameters. Following prior work [29], [30], we set $Q = 4$ and $B = 2$ for SAX, and $\alpha = 1$ for the loss function. All hyperparameter configurations are provided in Appendix C of the supplemental material.

Benchmarked Methods. For non-transformer methods, we choose ROCKET [41] as a representative SOTA baseline. We also compare with two SOTA shape-based transformers (ShapeFormer and PatchTST) [19], [42], and five representative time series transformers, covering both the vanilla transformer [7]($O(N^2)$) and state-of-the-art efficient variants: Informer [23] ($O(N \log N)$), Performer [43] ($O(N)$), Fedformer [44] ($O(N)$), and DARKER [25] ($O(N)$).

[2]https://github.com/rdzuo/tsketch

B. Overall Evaluation

We conduct a comprehensive evaluation of TSketch on its efficiency (Section IV-B1), scalability (Section IV-B2), and accuracy (Section IV-B3), demonstrating its superior performance over baselines.

1) Efficiency evaluation: We evaluate the training efficiency of TSketch by reporting the end-to-end training time across all datasets. For each method, we compare two input settings: (1) using raw time series values (*Baseline*) and (2) using TSketch tokens (*Ours*). Figure 10 presents the results, ordered by ascending sequence length N.

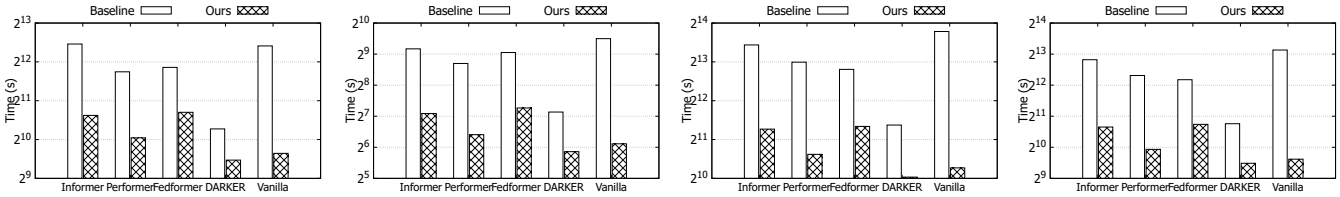
Across all datasets, TSketch consistently reduces training time by achieving speedups of $2\times$–$120\times$ compared to the baselines. For example, on *DucksAndGeese*, Fedformer requires more than 2^{10} seconds to train, whereas with TSketch the training time drops below 2^5 seconds. Further, we observe that the magnitude of acceleration varies depending on the base model's complexity. For instance, TSketch accelerates the computationally intensive vanilla transformer by $7\times$–$120\times$. Even for SOTA efficient models like DARKER, TSketch still provides a $2\times$–$40\times$ speed up. This also demonstrates TSketch's broad compatibility, as it can be effectively integrated with both traditional and highly optimized transformer models to further boost their efficiency.

The speedup becomes more pronounced as the length of the time series increases. For example, with the vanilla Transformer, the acceleration scales from $7\times$ on RightWhaleCalls ($N = 4,767$) to $120\times$ on BinaryHeartBeat ($N = 18,530$), underscoring our method's strength in handling long time series. TSketch not only accelerates training but also enables models to run in scenarios where baselines fail due to GPU memory overflow. Notably, Informer and vanilla Transformers encounter GPU OOM errors on *DucksAndGeese* and *UrbanSound*, but their TSketch-enhanced versions successfully complete training. Since ROCKET is CPU-based and not directly comparable with GPU-trained transformers, we provide additional ROCKET comparisons in Appendix D, which further confirm the efficiency of TSketch on long time series, consistent with recent findings [45], [46].

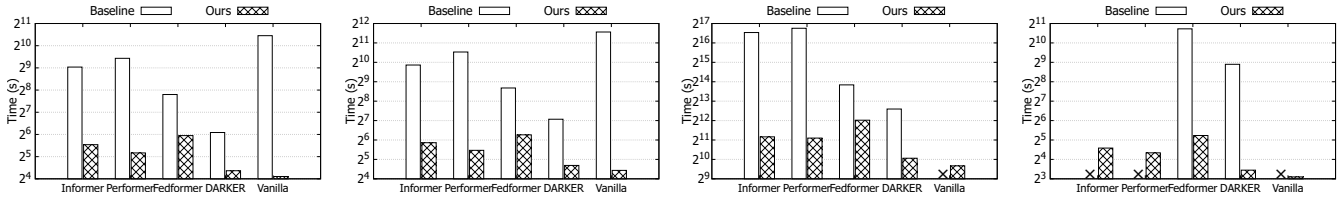
Overall, these results demonstrate that TSketch provides both efficiency and scalability: it not only reduces training time substantially but also extends the applicability of transformer-based models to previously infeasible long-sequence datasets.

2) Scalability analysis: To further investigate scalability, we evaluate the training time of using raw time series values and TSketch on synthetic datasets with input lengths N systematically varied from 2^9 to 2^{14}.

From Figure 11, we can observe that the training time for the baseline models grows with the input length N, consistent with their $O(N)$ or higher computational complexity. In contrast, in our method, we compute the K TSketches for the time series as input to the transformer. Consequently, the training time becomes independent of N and remains almost constant across all datasets, demonstrating our method's ability to handle arbitrarily long time series datasets with a fixed computational budget.

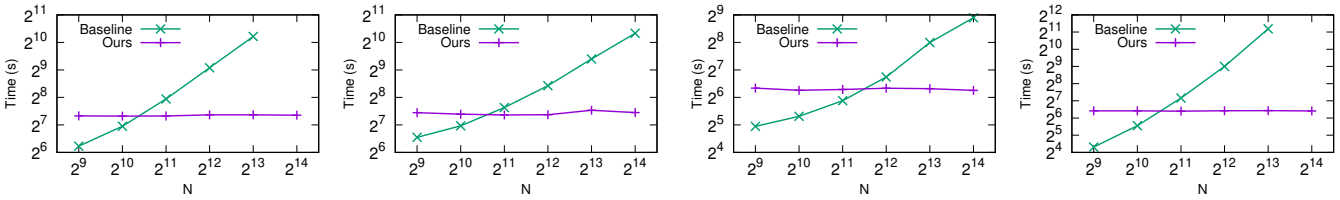


(a) RightWhaleCalls (b) KeplerLightCurves (c) FruitFlies (d) FaultDetectionA



(e) CatsDogs (f) BinaryHeartbeat (g) UrbanSound (h) DucksAndGeese

Fig. 10. Training time on 8 datasets. The symbol "×" denotes a GPU out-of-memory (OOM) error, which occurred when a model could not process an excessively long input sequence.



(a) Informer (b) Fedformer (c) DARKER (d) Vanilla transformer

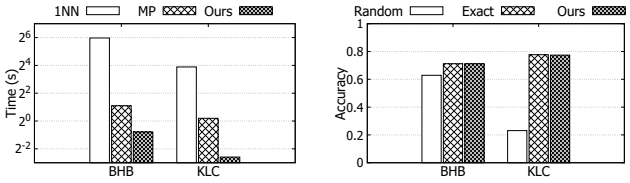
Fig. 11. Training time on varying the input length N on the synthetic dataset

TABLE V
ACCURACY OF OUR METHOD, BENCHMARKED METHODS, AND ADDITIONAL BASELINES (ROCKET, SHAPEFORMER, PATCHTST) ON 8 UCR DATASETS.

Dataset	ROCKET	ShapeFormer	PatchTST	Informer		Performer		Fedformer		DARKER		Vanilla Transformer	
				Baseline	**Ours**	Baseline	**Ours**	Baseline	**Ours**	Baseline	**Ours**	Baseline	**Ours**
RightWhaleCalls	**0.702**	0.516	0.512	0.503	0.507	0.501	0.504	0.506	0.508	0.501	0.508	0.532	0.523
KeplerLightCurves	0.717	0.759	0.716	0.391	**0.779**	0.303	0.704	0.318	0.764	0.351	0.774	0.474	0.777
FruitFlies	0.787	0.765	0.613	0.557	0.787	0.544	0.762	0.545	**0.797**	0.625	0.781	0.674	0.790
FaultDetectionA	0.914	0.946	0.873	0.447	0.931	0.424	0.889	0.448	0.901	0.446	0.906	0.457	**0.948**
CatsDogs	0.634	0.603	0.561	0.500	0.646	0.445	0.554	0.482	0.634	0.500	**0.701**	0.494	0.652
BinaryHeartbeat	0.585	0.698	0.654	0.707	0.726	0.668	0.731	0.683	**0.731**	0.717	0.727	0.697	0.712
UrbanSound	**0.450**	0.131	0.124	0.122	0.156	0.123	0.124	0.133	0.153	0.142	0.130	N/A	0.307
DucksAndGeese	0.540	0.660	N/A	N/A	0.680	N/A	0.480	0.240	0.680	0.240	0.640	N/A	**0.700**
Average Rank	4.875	5.250	8.750	10.375	3.500	12.250	7.250	9.938	3.750	9.062	4.562	9.062	2.375
p-value	—	—	—	—	0.008	—	0.039	—	0.008	—	0.039	—	0.016

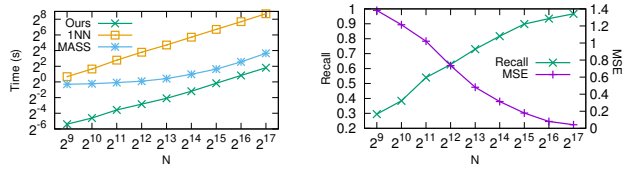
3) Accuracy results: Table V reports the classification accuracy on eight datasets using five transformer models, comparing each method with its TSketch-enhanced counterpart. Underlines indicate the higher accuracy between the baseline (using raw values) and our method (using TSketch). To provide a broader comparison, we also include one SOTA non-transformer method (ROCKET) and two SOTA shape-based transformer methods (Shapeformer and PatchTST).

Overall, TSketch consistently improves accuracy across almost all datasets and transformer models. This verifies that TSketch provides more informative inputs than value-based tokens. In addition to absolute accuracy gains, TSketch consistently improves the relative ranking of all transformer models. Importantly, models such as Informer and Fedformer achieve large rank reductions when using our proposed TSketches. Also, by incorporating TSketch, almost all transformer models, such as Informer, Fedformer, DARKER, and vanilla transformer, outperform ROCKET, ShapeFormer, and PatchTST in terms of average accuracy. Moreover, TSketch frequently enables a transformer model to achieve the best performance on individual datasets, such as on *CatsDogs* and *KeplerLightCurves*. Statistical tests further confirm that these improvements are significant, with all models achieving p-values below 0.05. Finally, we observe that vanilla



(a) Search Time (b) Accuracy

Fig. 12. Performance of TSKETCHER in real datasets.



(a) Search Time (b) Recall and MSE

Fig. 13. Performance of TSKETCHER varying input lengths.

transformer+TSketch attains the best average rank overall. Combined with its competitive training efficiency (Figure 10), this result highlights that a simple transformer equipped with TSketch can be both effective and efficient for long time series.

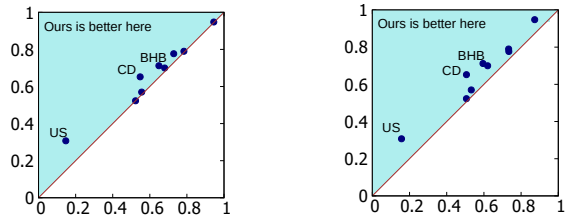
C. Evaluation of TSKETCHER.

We further evaluate the accuracy, efficiency, and scalability of TSKETCHER using two representative datasets: *Binary-Heartbeat* (BHB) and *KeplerLightCurves* (KLC).

Accuracy of TSKETCHER. We compare TSKETCHER with two alternatives: (i) randomly selecting K subsequences from the time series as input; and (ii) taking the exact nearest neighbor of each prototype as input. Figure 12(b) shows that in both datasets, our approach matches the accuracy of exact search, and consistently outperforms random selection, demonstrating its ability to efficiently identify TSketch without compromising accuracy.

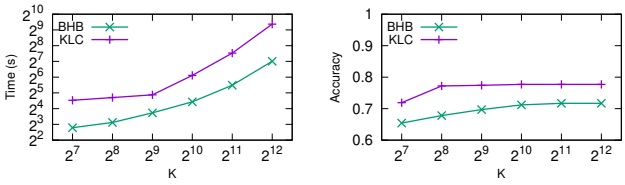
Efficiency of TSKETCHER. To measure efficiency, we record the time required to search TSketch for each prototype given an input time series. We compare against two baselines: (i) direct 1-NN search, a naive approach in existing shape-based transformers [19], [24], and (ii) MASS [47], a widely adopted method for fast subsequence matching. As shown in Figure 12(a), TSKETCHER achieves up to $4\times$ speedup over MASS and over $60\times$ compared with 1-NN.

Scalability Analysis of TSKETCHER. To further investigate scalability, we use synthetic datasets with input lengths N ranging from 2^9 to 2^{17}. As illustrated in Figure 13(a), the search time of TSKETCHER grows more slowly than 1-NN and MASS. At $N = 2^{17}$, it is more than two orders of magnitude faster than 1-NN and nearly one order faster than MASS. Figure 13(b) shows the recall and MSE of TSKETCHER results compared to the exact 1-NN ground truth. As N increases, the recall of TSKETCHER improves, approaching 1.0 (indicating an exact search) at 2^{17}. Also, the MSE decreases as N increases. Both results demonstrate that TSKETCHER provides accurate search of TSketch, particularly for long time series.



(a) Ours vs. without kernels (b) Ours vs. triplet loss

Fig. 14. Accuracy comparison on STRATALOSS



(a) Training time on varying K (b) Accuracy on varying K

Fig. 15. Performance on varying K

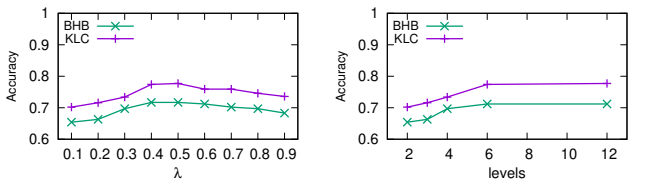
D. Experiment on STRATALOSS

We then conduct ablation studies to evaluate the effectiveness of STRATALOSS, with results illustrated in Figure 14. Figure 14(a) compares STRATALOSS with and without the convolution kernel term $\mathcal{L}_0$, showing consistent accuracy gains across datasets. This confirms that kernels help capture multi-scale variations and alleviate issues related to prototype scaling. Figure 14(b) compares STRATALOSS against the vanilla triplet loss, where the stratified margins lead to clear improvements by preserving relative similarity. Meanwhile, we highlight the three datasets with the most significant accuracy gains, *UrbanSound* (US), *CatsDogs* (CD), and *BinaryHeartbeat* (BHB), all of which have time series lengths exceeding $10,000$. This suggests that STRATALOSS is particularly effective for handling long time series. Overall, both studies demonstrate that STRATALOSS produces a robust and discriminative embedding space.

E. Parameter Sensitivity

In this subsection, we study the effect of three key parameters of TSTUDIO on BHB and KLC.

- **Number of TSketches (K).** Figure 15 shows that larger K substantially increases training cost, while accuracy stabilizes around $K = 1024$ for both datasets. We set $K = 1024$ by default, though other values may be tuned for specific applications.
- **Balancing weight (λ) in Equation 13.** As shown in Figure 16(a), accuracy peaks near $\lambda = 0.5$, confirming the benefit of jointly optimizing STRATALOSS and reconstruction loss.
- **Number of strata (m).** Figure 16(b) indicates that fewer strata degrade accuracy, while more strata consistently improve it. This validates the importance of multi-level stratification for learning richer representations.



(a) Accuracy on varying λ (b) Accuracy on varying the number of strata

Fig. 16. Parameter of STRATALOSS

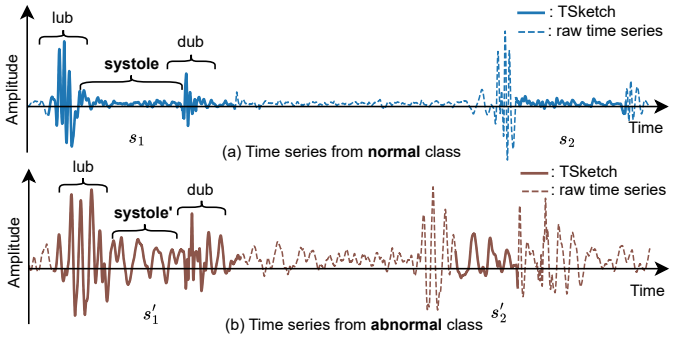


Fig. 17. TSketches of two time series from BinaryHeartbeat.

F. A Case Study on BinaryHeartbeat

To validate that our proposed TSketch effectively captures semantic information, we conduct a case study on the *BinaryHeartbeat* dataset, with heart sound recordings labeled as *normal* or *abnormal*. Each recording is a univariate time series of length 18,530.

We randomly select one sample from each class. The raw time series is shown as dashed lines in Figure 17, while the solid lines represent the TSketches discovered by our method. First, our method discovers TSketches that naturally correspond to the 'lub', 'dub', and 'systole' phases of a heartbeat cycle–correspondence validated by both medical knowledge and a public LLM. Consequently, the original time series is effectively represented as a repeating sequence of these three sketches. Meanwhile, the TSketches of the two samples show a marked divergence in their systolic phases: the heart sound amplitudes for the normal sample (*systole*) are low and stable, while those for the abnormal sample, characterized by a *systolic murmur* and labeled *systole'*, are high and volatile. These findings demonstrate that our proposed TSketch not only carries rich semantic information but also reduces redundancy by discarding repeated cycles, thus improving both the efficiency and accuracy of the transformers. In addition, for TSketches s_2 and s_2', they carry the same semantic meaning but are different at scales, which illustrates that the $\mathcal{L}_{sc}$ in STRATALOSS successfully handles the *global scaling*. We present an additional case study in Appendix E to further validate that the TSketches obtained from dataset FaultDectionA contain meaningful semantic information.

V. RELATED WORK

In this section, we first briefly review the existing transformers for time series, especially the efficient ones. Furthermore, we report the transformers that operate on time series subsequences, highlighting their unique representational advantages.

Efficient Transformers for Time Series. Transformer-based models have been widely used for time series, demonstrating superior performance in various tasks, including classification [17], [19], imputation [48]–[50], and forecasting [18], [42]. Despite their success, these models inherently have the quadratic complexity of the attention mechanism [34], [51].

A large body of work has focused on improving the efficiency of time series transformers. Informer [23] introduces a ProbSparse attention mechanism that exploits the sparsity of attention weights, reducing the complexity of long-sequence forecasting to $O(N \log N)$. Fedformer [44] computes the attention in the frequency domain via Fourier and wavelet transforms, achieving linear complexity by operating on a fixed subset of frequency components. Despite these advances, existing methods share a fundamental limitation: their input tokens are raw values at each timestamp. As a result, the input sequence length N remains unchanged, which subsequently adversely affects the efficiency, especially for long time series.

Shape-based Transformers. Classical time series classification methods have exploited shape-based representations, including dictionary-based methods that encode subsequences into symbolic words [52], [53] and shapelet-based methods [5] that identify discriminative subsequences [54]. Inspired by their ideas, recent research explores tokenizing time series into semantic subsequences, rather than individual timestamps, to improve transformer performance [24], [55], [56]. PatchTST [42] adopts fixed-size patches as tokens, but its rigid windowing can misalign with intrinsic patterns. SVP-T [24] selects representative subsequences (shapes) near cluster centroids to serve as tokens. DARKER [25] proposes a data-driven kernel-based attention mechanism, tailoring attention to shape information in time series. Shapeformer [19] discovers shapelets offline and feeds the best-matching subsequences into the transformer. Despite these advances, the process of selecting representative subsequences is computationally costly and does not consider global scaling.

VI. CONCLUSION

In this paper, we propose TSketch, a new time series primitive for efficient transformer models, and TSTUDIO, a framework that extracts them. TSTUDIO first organizes subsequences via SAX-LSH and an SAX bucket distance with a probabilistic order-preserving (POP) guarantee, then learns representations with STRATALOSS, and finally uses TSKETCHER to select the approximate best matching per prototype as TSketches. Our experiments on eight datasets demonstrate that replacing value tokens with TSketches not only significantly reduces training time for time series transformers but also achieves superior or highly competitive accuracy. In the future, we plan to extend the application of TSketch to other downstream tasks, such as forecasting and anomaly detection. Furthermore, we intend to employ LLMs to verify the semantic meaning of our discovered TSketches on a broader scale.

AI/LLM USAGE DISCLOSURE

ChatGPT was used to polish the language for better readability. All technical contributions, including the proposed method, analysis, and experiments, were developed and verified by the authors.

REFERENCES

[1] P. Boniol, M. Meftah, E. Remy, and T. Palpanas, "dcam: Dimension-wise class activation map for explaining multivariate data series classification," in *ACM SIGMOD*, 2022, pp. 1175–1189.

[2] D. Kieu, T. Kieu, P. Han, B. Yang, C. S. Jensen, and B. Le, "Team: Topological evolution-aware framework for traffic forecasting–extended version," *Proc. VLDB Endow.*, vol. 18, no. 2, pp. 265–278, 2024.

[3] N. Mohammadi Foumani, L. Miller, C. W. Tan, G. I. Webb, G. Forestier, and M. Salehi, "Deep learning for time series classification and extrinsic regression: A current survey," *ACM Computing Surveys*, vol. 56, no. 9, pp. 1–45, 2024.

[4] A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, "The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances," *Data mining and knowledge discovery*, vol. 31, pp. 606–660, 2017.

[5] J. Hills, J. Lines, E. Baranauskas, J. Mapp, and A. Bagnall, "Classification of time series by shapelet transformation," *Data mining and knowledge discovery*, vol. 28, pp. 851–881, 2014.

[6] Y. Hao and H. Cao, "A new attention mechanism to classify multivariate time series." in *IJCAI*, 2020, pp. 1999–2005.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017.

[8] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *NAACL*, 2019.

[9] H. Peng, N. Pappas, D. Yogatama, R. Schwartz, N. Smith, and L. Kong, "Random feature attention," in *ICLR*, 2021.

[10] J. H. Clark, D. Garrette, I. Turc, and J. Wieting, "Canine: Pre-training an efficient tokenization-free encoder for language representation," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 73–91, 2022.

[11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NeurIPS 2020. Red Hook, NY, USA: Curran Associates Inc., 2020.

[12] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," *ArXiv*, vol. abs/2302.13971, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:257219404

[13] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.

[14] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[15] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," *arXiv preprint arXiv:1911.02116*, 2019.

[16] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.

[17] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," in *ACM SIGKDD*, 2021, p. 2114–2124.

[18] Y. Cheng, C. Guo, B. Yang, H. Yu, K. Zhao, and C. S. Jensen, "A memory guided transformer for time series forecasting," *Proc. VLDB Endow.*, vol. 18, no. 2, pp. 239–252, 2024.

[19] X.-M. Le, L. Luo, U. Aickelin, and M.-T. Tran, "Shapeformer: Shapelet transformer for multivariate time series classification," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 1484–1494.

[20] Y. Li, X. Lu, H. Xiong, J. Tang, J. Su, B. Jin, and D. Dou, "Towards long-term time-series forecasting: Feature, pattern, and distribution," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 1611–1624.

[21] M. Wang, J. Yang, B. Yang, H. Li, T. Gong, B. Yang, and J. Cui, "Towards lightweight time series forecasting: A patch-wise transformer with weak data enriching," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 1278–1291.

[22] S. Tuli, G. Casale, and N. R. Jennings, "Tranad: Deep transformer networks for anomaly detection in multivariate time series data," *PVLDB*, pp. 1201–1214, 2022.

[23] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *AAAI*, 2021, pp. 11 106–11 115.

[24] R. Zuo, G. Li, B. Choi, S. S Bhowmick, D. N.-Y. Mah, and G. L.-H. Wong, "Svp-t: A shape-level variable-position transformer for multivariate time series classification," in *AAAI*, 2023, pp. 11 497–11 505.

[25] R. Zuo, G. Li, R. Cao, B. Choi, J. Xu, and S. S. Bhowmick, "Darker: Efficient transformer with data-driven attention mechanism for time series," *Proceedings of the VLDB*, vol. 17, no. 11, pp. 3329–3242, 2024.

[26] G. Li, B. Choi, J. Xu, S. S. Bhowmick, D. N.-y. Mah, and G. L. Wong, "Ips: Instance profile for shapelet discovery for time series classification," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 1781–1793.

[27] G. Li, B. Choi, J. Xu, S. S. Bhowmick, K.-P. Chun, and G. L. Wong, "Shapenet: A shapelet-neural network approach for multivariate time series classification," in *AAAI*, 2021, pp. 8375–8383.

[28] H. Xiong, H. Zhang, Z. Wang, Z. He, P. Wang, and X. S. Wang, "Civet: Exploring compact index for variable-length subsequence matching on time series," *Proc. VLDB Endow.*, vol. 17, no. 9, p. 2123–2135, May 2024.

[29] G. Li, B. Choi, R. Zuo, S. S. Bhowmick, and J. Xu, "lesax index: A learned sax representation index for time series similarity search," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 2025, pp. 1995–2008.

[30] A. Camerra, T. Palpanas, J. Shieh, and E. Keogh, "isax 2.0: Indexing and mining one billion time series," in *2010 IEEE International Conference on Data Mining*. IEEE, 2010, pp. 58–67.

[31] K. Echihabi, "High-dimensional vector similarity search: From time series to deep network embeddings," in *ACM SIGMOD*, D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, Eds., 2020, pp. 2829–2832.

[32] Q. Wang and T. Palpanas, "Deep learning embeddings for data series similarity search," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 1708–1716.

[33] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The ucr time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[34] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–28, 2022.

[35] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *ICLR*, 2021.

[36] J.-Y. Franceschi, A. Dieuleveut, and M. Jaggi, "Unsupervised scalable representation learning for multivariate time series," *Advances in neural information processing systems*, vol. 32, 2019.

[37] J. Shieh and E. Keogh, "isax: indexing and mining terabyte sized time series," in *Proceedings of the 14th ACM SIGKDD international*

conference on Knowledge discovery and data mining, 2008, pp. 623–631.

[38] J. Lin, E. Keogh, L. Wei, and S. Lonardi, "Experiencing sax: a novel symbolic representation of time series," *Data Mining and knowledge discovery*, vol. 15, pp. 107–144, 2007.

[39] L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. N. Bennett, J. Ahmed, and A. Overwijk, "Approximate nearest neighbor negative contrastive learning for dense text retrieval," in *International Conference on Learning Representations*, 2021.

[40] G. Biau and L. Devroye, *Lectures on the nearest neighbor method*. Springer, 2015, vol. 246.

[41] A. Dempster, F. Petitjean, and G. I. Webb, "Rocket: exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020.

[42] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *ICLR*, 2023.

[43] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser *et al.*, "Rethinking attention with performers," in *ICLR*, 2021.

[44] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *International conference on machine learning*. PMLR, 2022, pp. 27 268–27 286.

[45] N. M. Foumani, C. W. Tan, G. I. Webb, and M. Salehi, "Improving position encoding of transformers for multivariate time series classification," *Data Min. Knowl. Discov.*, vol. 38, no. 1, p. 22–48, Sep. 2023. [Online]. Available: https://doi.org/10.1007/s10618-023-00948-2

[46] N. Mohammadi Foumani, L. Miller, C. W. Tan, G. I. Webb, G. Forestier, and M. Salehi, "Deep learning for time series classification and extrinsic regression: A current survey," *ACM Comput. Surv.*, vol. 56, no. 9, Apr. 2024. [Online]. Available: https://doi.org/10.1145/3649448

[47] S. Zhong and A. Mueen, "Mass: distance profile of a query over a time series," *Data Min. Knowl. Discov.*, vol. 38, no. 3, p. 1466–1492, Feb. 2024.

[48] J.-S. Hong, J. Yao, J. Mueller, and J.-L. Wang, "SAND: Smooth imputation of sparse and noisy functional data with transformer networks," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[49] J. Xu, F. Lyu, and P. C. Yuen, "Density-aware temporal attentive stepwise diffusion model for medical time series imputation," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM 2023, Birmingham, United Kingdom, October 21-25, 2023*, I. Frommholz, F. Hopfgartner, M. Lee, M. Oakes, M. Lalmas, M. Zhang, and R. L. T. Santos, Eds. ACM, 2023, pp. 2836–2845.

[50] Z. Lai, D. Zhang, H. Li, D. Zhang, H. Lu, and C. S. Jensen, "Rectsi: Resource-efficient correlated time series imputation via decoupled pattern learning and completeness-aware attentions," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1474–1483.

[51] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun, "Transformers in time series: A survey," *IJCAI*, 2023.

[52] J. Lin, R. Khade, and Y. Li, "Rotation-invariant similarity in time series using bag-of-patterns representation," *Journal of Intelligent Information Systems*, vol. 39, pp. 287–315, 2012.

[53] P. Senin and S. Malinchik, "Sax-vsm: Interpretable time series classification using sax and vector space model," in *2013 IEEE 13th international conference on data mining*. IEEE, 2013, pp. 1175–1180.

[54] M. Middlehurst, P. Schäfer, and A. Bagnall, "Bake off redux: a review and experimental evaluation of recent time series classification algorithms: M. middlehurst et al." *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 1958–2031, 2024.

[55] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "itransformer: Inverted transformers are effective for time series forecasting," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=JePfAI8fah

[56] Y. Liu, G. Qin, X. Huang, J. Wang, and M. Long, "Timer-XL: Long-context transformers for unified time series forecasting," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=KMCJXjlDDr

[57] C. M. Bishop and N. M. Nasrabadi, *Pattern recognition and machine learning*. Springer, 2006, vol. 4, no. 4.

APPENDIX

A. Proof of Theorem: Probabilistic Order-Preserving Property of SAX Bucket Distance

Definition 2. ***(SAX distance (MinDist) [38])*** *Given two time series subsequences $T = (t_1, \ldots, t_n), U = (u_1, \ldots, u_n)$, and their corresponding SAX representations $b_T = (b_T^1, \ldots, b_T^w), b_U = (b_U^1, \ldots, b_U^w)$, where w is the number of PAA segments. Assume each pair b_T^k and b_U^k maps to value intervals $[x_t, y_t]$ and $[x_u, y_u]$, respectively. The distance between the k-th segments is defined as:*

$$dist_i(b_T^k, b_U^k) = \begin{cases} 0, & \text{if } b_T^k = b_U^k \\ \min(|x_t - y_u|, |x_u - y_t|), & \text{else.} \end{cases} \tag{24}$$

Then the distance between two SAX representations is computed as:

$$MinDist(b_T, b_U) \equiv \sqrt{\frac{n}{w}} \sqrt{\sum_{i=1}^{w} \left(dist(b_T^k, b_U^k)\right)^2} \tag{25}$$

Theorem 3 (Probabilistic Order-Preserving Property of SAX Bucket Distance). *Let w be a positive integer, and let $T = (t_1, \ldots, t_n)$ and $U = (u_1, \ldots, u_n)$ be two time series subsequences such that $t_1, \ldots, t_n, u_1, \ldots, u_n \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$.*

Assume their SAX Bucket Distance is $iBDIST(b_T, b_U) = d_{iB}$. Then, for any $\epsilon, \delta > 0$, the following inequality holds:

$$\begin{aligned} &\Pr[\|T - U\|_2^2 \le n\delta^2 + d_{iB}\frac{n}{w}(d_z^2 + 2\delta d_z) + 2(n - w) + \epsilon] \\ &\ge (2\Phi(\sqrt{\frac{n}{2w}}(d_z + \delta)) - 1)^{d_{iB}} \cdot (2\Phi(\sqrt{\frac{n}{2w}}\max(\delta, d_z) - 1))^{w - d_{iB}} \\ &\quad \cdot \frac{\epsilon^2}{8n(1 - \frac{w}{n})^2 + \epsilon^2} \end{aligned} \tag{26}$$

where $d_z = \Phi^{-1}(1 - \frac{1}{2^B}) - \Phi^{-1}(1 - \frac{2}{2^B})$ is a constant, $\Phi(\cdot)$ is CDF of the standard Gaussian distribution and B is the number of SAX bits per segment.

Proof.

$$\begin{aligned} \|T - U\|_2^2 &= \sum_{i=1}^{w}\sum_{j=1}^{\frac{n}{w}}(t_{ij} - u_{ij})^2 \\ &= \sum_{i=1}^{w}\{\sum_{j=1}^{\frac{n}{w}}[(t_{ij} - \bar{t_i}) - (u_{ij} - \bar{u_i}) + (\bar{t_i} - \bar{u_i})]^2\} \\ &= \frac{n}{w}\sum_{i=1}^{w}(\bar{t_i} - \bar{u_i})^2 + \sum_{i=1}^{w}\sum_{j=1}^{\frac{n}{w}}[(t_{ij} - \bar{t_i}) - (u_{ij} - \bar{u_i})]^2 \end{aligned} \tag{27}$$

Let $A = \frac{n}{w}\sum_{i=1}^{w}(\bar{t_i} - \bar{u_i})^2$ and $G = \sum_{i=1}^{w}\sum_{j=1}^{\frac{n}{w}}[(t_{ij} - \bar{t_i}) - (u_{ij} - \bar{u_i})]^2$. Assume $iBDIST(b_T, b_U) = d_{iB}$ and it indicates that at most d_{iB} segments in the SAX representations of T and U are different. Under the circumstance, we first calculate $|\bar{t_i} - \bar{u_i}|$ where t_i and u_i lie in adjacent interval in each segment. Let $\{z_i\}_{i=1}^{2^B-1}$ be the quantile points in each segment and assume $|\bar{t_i} - \bar{u_i}| \le |z_{2^B-1} - z_{2^B-2}| + \delta$ where $\delta > 0$. According to the property of SAX, $d_z = |z_{2^B-1} - z_{2^B-2}| = \Phi^{-1}(1 - \frac{1}{2^B}) - \Phi^{-1}(1 - \frac{2}{2^B})$ where $\Phi(\cdot)$ is the cumulative distribution function of the Gaussian distribution. Additionally, since $\bar{t_i} - \bar{u_i} \sim \mathcal{N}(0, \frac{2w}{n})$, $\forall \delta > 0$, we have the inequality below:

$$\Pr[|\bar{t_i} - \bar{u_i}| \le d_z + \delta] = 2\Phi(\sqrt{\frac{n}{2w}}(d_z + \delta)) - 1 \tag{28}$$

Therefore, we can calculate A:

$$\begin{aligned} A &= \frac{n}{w}\sum_{i=1}^{w}(\bar{t_i} - \bar{u_i})^2 \\ &\le D_A(d_{iB}, \delta) = \frac{n}{w}(d_{iB} \cdot (d_z + \delta)^2 + (w - d_{iB}) \cdot \max(\delta, d_z)^2) \\ &= n \cdot \max(\delta, d_z)^2 + d_{iB} \cdot \frac{n}{w}(\min(\delta, d_z)^2 + 2\delta d_z) \end{aligned} \tag{29}$$

There exists:

$$\begin{aligned} &\Pr[A \le n \cdot \max(\delta, d_z)^2 + d_{iB} \cdot \frac{n}{w}(\min(\delta, d_z)^2 + 2\delta d_z)] \\ &\ge (2\Phi(\sqrt{\frac{n}{2w}}(d_z + \delta)) - 1)^{d_{iB}} \cdot (2\Phi(\sqrt{\frac{n}{2w}}\max(\delta, d_z) - 1))^{w - d_{iB}} \end{aligned} \tag{30}$$

Then we analyze the distribution of G. Since $t_{ij} \sim \mathcal{N}(0, 1)$, we obtain $\mathbb{E}[t_{ij} - \bar{t_i}] = 0$ and $\mathrm{Var}(t_{ij} - \bar{t_i})$ as follows.

$$\begin{aligned} \mathrm{Var}(t_{ij} - \bar{t_i}) &= \mathrm{Var}(t_{ij}) + \mathrm{Var}(\bar{t_i}) - 2\mathrm{Cov}(t_{ij}, \bar{t_i}) \\ &= 1 + \mathrm{Var}(\frac{w}{n}\sum_{k=1}^{\frac{n}{w}} t_{ik}) - 2\mathrm{Cov}(t_{ij}, \frac{w}{n}\sum_{k=1}^{\frac{n}{w}} t_{ik}) \\ &= 1 + (\frac{w}{n})^2\sum_{k=1}^{\frac{n}{w}}\mathrm{Var}(t_{ik}) - 2\frac{w}{n}\sum_{k=1}^{\frac{n}{w}}\mathrm{Cov}(t_{ij}, t_{ik}) \\ &= 1 + (\frac{w}{n})^2 \cdot \frac{n}{w} - 2\frac{w}{n} = 1 - \frac{w}{n} \end{aligned} \tag{31}$$

Thus it yields $(t_{ij} - \bar{t_i}), (u_{ij} - \bar{u_i}) \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1 - \frac{w}{n})$. Let $z_{ij} = (t_{ij} - \bar{t_i}) - (u_{ij} - \bar{u_i})$, it holds $z_{ij} \sim \mathcal{N}(0, 2(1 - \frac{w}{n}))$. Then $\sum_{j=1}^{\frac{n}{w}} \frac{z_{ij}^2}{2(1-\frac{w}{n})} \sim \chi^2(\frac{n}{w})$ and $\frac{B}{2(1-\frac{w}{n})} = \sum_{i=1}^{w}\sum_{j=1}^{\frac{n}{w}} \frac{z_{ij}^2}{2(1-\frac{w}{n})} \sim \chi^2(n)$. We obtain $\mathbb{E}[G] = 2(1 - \frac{w}{n}) \cdot n = 2(n - w)$ and $\mathrm{Var}(G) = 8n(1 - \frac{w}{n})^2$. According to the one-sided Chebyshev inequality, for $\forall \epsilon > 0$, it holds

$$\Pr[G - 2(n - w) \ge \epsilon] \le \frac{8n(1 - \frac{w}{n})^2}{8n(1 - \frac{w}{n})^2 + \epsilon^2}. \tag{32}$$

Conversely, we have

$$\Pr[G \le 2(n - w) + \epsilon = D_B(\epsilon)] \ge \frac{\epsilon^2}{8n(1 - \frac{w}{n})^2 + \epsilon^2} \tag{33}$$

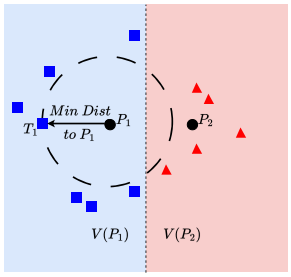
After combining Equation 30 and 33, it yields

$$\begin{aligned} &\Pr[\|T - U\|_2^2 = A + G \le D_A(d_{iB}, \delta) + D_G(\epsilon)] \\ &\ge (2\Phi(\sqrt{\frac{n}{2w}}(d_z + \delta)) - 1)^{d_{iB}} \cdot (2\Phi(\sqrt{\frac{n}{2w}}\max(\delta, d_z) - 1))^{w - d_{iB}} \\ &\quad \cdot \frac{\epsilon^2}{8n(1 - \frac{w}{n})^2 + \epsilon^2} \end{aligned} \tag{34}$$

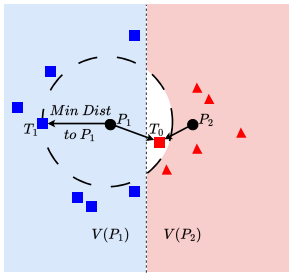
□

B. Extra Explanation of Theorem: Lower bound of Algorithm 3

Theorem 4. (Lower bound of Algorithm 3) *Given time series prototypes $\mathcal{P} = \{P_1, \ldots, P_K\}$ and time series subsequences*



(a) Two Voronoi regions



(b) Two Voronoi regions with special area

Fig. 18. Illustration of two example distributions on a two-dimensional surface. If one subsequence lies in the white area, our method will produce the wrong NN result.

$\mathcal{T} = \{T_1, \ldots, T_N\}$ where $P_1, \ldots, P_K, T_1, \ldots, T_N \overset{\text{iid}}{\sim} \mathcal{N}(0, I_d)$ where I_d denotes the $d \times d$ identity matrix, TSketch s_i returned by Algorithm 3 has the following property:

$$\Pr\left[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)\right] \geq 1 - \exp\left(-\alpha(d) \cdot \frac{N}{K}\right), \tag{35}$$

where $\mathrm{NN}_A(x)$ denotes the nearest neighbor search (NN) of x in set A and $\alpha(d)$ is a positive constant dependent only on the dimension d.

Figure 18 illustrates this phenomenon by the relationships between some time series subsequences and two prototypes. Here, $V(P_i)$ denotes the Voronoi region of prototype P_i, as defined in the proof of Theorem 2. When the Voronoi regions of two prototypes overlap (the white area in the figure), a query subsequence located in this intersection may be assigned to its second-nearest prototype rather than the true nearest one.

Our theorem is proved based on the condition that the prototypes and time series subsequences are independent and identically Gaussian distributed. By using a sufficient number of Gaussians and by adjusting their means and covariances as well as the coefficients in the linear combination, almost any continuous density can be approximated to arbitrary accuracy [57]. Accordingly, our probability in Equation 35 and the proof can still be utilized to obtain lower bounds under the circumstance of Gaussian mixture models. Thus, the probability proves the feasibility of our approximate nearest neighbor search method since the dataset may not conform to the specific distribution in practice.

Theorem 5. (Lower bound of Algorithm 3 of Distribution under GMM) *Assume two Gaussian mixture models GMM_P and GMM_T with their corresponding probability functions*

$$p_P(x) = \sum_{k=1}^{K_P} \pi_{P_k} \mathcal{N}(x; \boldsymbol{\mu}_{P_k}, \boldsymbol{\Sigma}_{P_k}) \tag{36}$$

and

$$p_T(x) = \sum_{l=1}^{K_T} \pi_{T_l} \mathcal{N}(x; \boldsymbol{\mu}_{T_l}, \boldsymbol{\Sigma}_{T_l}) \tag{37}$$

where π_{P_k}, π_{T_l} are mixing coefficients and $\sum_{k=1}^{K_P} \pi_{P_k} = \sum_{l=1}^{K_T} \pi_{T_l} = 1$.

Given time series prototypes $\mathcal{P} = \{P_1, \ldots, P_K\}$ and time series subsequences $\mathcal{T} = \{T_1, \ldots, T_N\}$ where $P_1, \ldots, P_K \overset{\text{iid}}{\sim} GMM_P$ and $T_1, \ldots, T_N \overset{\text{iid}}{\sim} GMM_T$. We denote the nearest neighbor search of x on set A as $\mathrm{NN}_A(x)$. It holds,

$$\Pr[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)] \geq 1 - \sum_{k=1}^{K_P} \pi_{P_k} \cdot \exp\left(-\beta(d,k) \cdot \frac{\pi_{T_k}}{\pi_{P_k}} \cdot \frac{N}{K}\right) \tag{38}$$

where sketch s_i is obtained in Algorithm 3, β is a constant, N and K denote the size of $\mathcal{T}$ and $\mathcal{P}$.

Proof. Similar to the proof of Theorem 2, we still explore the probability below first:

$$\Pr[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)] = \mathbb{E}_{\mathcal{T},\ \mathcal{P}}\left[\frac{1}{K}\sum_{i=1}^{K} \mathbf{1}\left(\mathrm{NN}_{\mathcal{P}}\left(\mathrm{NN}_{\mathcal{T}}(P_i)\right) = P_i\right)\right] \tag{39}$$

where $\mathbf{1}$ denote the indicator function. Besides, we also have the Voronoi region of P_i denoted as $V(P_i) = \{x \in \mathbb{R}^d | \|x - P_i\| \leq \|x - P_j\|, \forall j \neq i\}$. For each P_i, we explore its nearest neighbor $T^* = \mathrm{NN}_{\mathcal{T}}(P_i)$. If $T^* \in V(P_i)$, we have $p_i = \mathrm{NN}_{\mathcal{T}}(P_i)$. Thus the probability holds,

$$\begin{aligned}
\Pr[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)] &= \mathbb{E}_{\mathcal{T},\ \mathcal{P}}\left[\frac{1}{K}\sum_{i=1}^{K} \mathbf{1}\left(T^* \in V(P_i)\right)\right] \\
&= \sum_{k=1}^{K_P}\sum_{l=1}^{K_T} \Pr[P_i \in \mathcal{E}_k, T \in \mathcal{F}_l] \cdot \Pr[T^* \in V(P_i) | P_i \in \mathcal{E}_k, T \in \mathcal{F}_l] \\
&= \sum_{k=1}^{K_P}\sum_{l=1}^{K_T} \pi_{P_k}\pi_{T_l} \Pr[T^* \in V(P_i) | P_i \in \mathcal{E}_k, T \in \mathcal{F}_l]
\end{aligned} \tag{40}$$

where $\mathcal{E}_k$ denotes P comes from the k-th Gaussian component and $\mathcal{F}_l$ denotes T comes from the ℓ-th Gaussian component. If $k = \ell$, the problem degrades into the probability under a single Gaussian distribution, similar to Theorem 4. Thus, we obtain:

$$P_{intra}^{(k)} \geq 1 - \exp\left(-\beta(d,k) \cdot \frac{N_k}{K_k}\right) \tag{41}$$

where $\beta(d,k)$ is a constant relying on dimension d and component k. N_k, K_k are the sample sizes of P, T lying into the same component, respectively. However, if $k \neq \ell$, we have:

$$P_{\text{cross}}^{(k,\ell)} = \Pr[\|P - T\| < \min_j \|P - T_j\| | P \in \mathcal{E}_k, T \in \mathcal{F}_\ell] \tag{42}$$

Then for $P \sim \mathcal{N}(\mu_{P_k}, \Sigma_{P_k}), T \sim \mathcal{N}(\mu_{T_\ell}, \Sigma_{T_\ell})$, we have $P - T \sim \mathcal{N}(\mu_{P_k} - \mu_{T_\ell}, \Sigma_{P_k} + \Sigma_{T_\ell})$. We can consider a simple case of Equation 42 where different components of GMM_P and GMM_T have spherical symmetric covariance for analysis. In such a case, the distribution of $\|P - T\|$ in Equation 42 can be degraded into a non-central χ^2 distribution. For the sake of convenience, we present some key steps of deriving $P_{\text{cross}}^{(k,\ell)}$. We can obtain the distribution of $\|P - T\|^2$ as follows:

$$\|z\|^2 \sim (\sigma_{P_k}^2 + \sigma_{T_\ell}^2) \cdot \chi_d^2(\lambda), \ \textit{where}\ \lambda = \frac{\|\mu_{P_k} - \mu_{T_\ell}\|^2}{\sigma_{P_k}^2 + \sigma_{T_\ell}^2}. \tag{43}$$

Thus, we obtain

$$\Pr[\|P - T\| \leq r | P \in \mathcal{E}_k, T \in \mathcal{F}_\ell] = F_{\chi_d^2}\left(\frac{r^2}{\sigma_{P_k}^2 + \sigma_{T_\ell}^2}\right) \tag{44}$$

The function in Equation 44 is a monotonically decreasing function with respect to r^2. In other words, a larger $\mu_{P_k} - \mu_{T_\ell}$ can lead to decreasing of $P_{\text{cross}}^{(k,\ell)}$, indicating that $P_{\text{cross}}^{(k,\ell)}$ can approximate to zero with obvious separated Gaussian compo-

TABLE VI
HYPERPARAMETERS

Dataset name	Batch size	Epochs
RightWhaleCalls	64	100
KeplerLightCurves	32	200
FruitFlies	16	100
FaultDetectionA	4	100
CatsDogs	4	100
BinaryHeartbeat	4	200
UrbanSound	64	50
DucksAndGeese	32	100

TABLE VII
RUNNING TIME COMPARISON ON EIGHT LONG UCR DATASETS.

Method	**RWC**	**KLC**	**FF**	**FDA**	**CD**	**BH**	**US**	**DAG**
ROCKET	**378.7**	**46.9**	1235.1	**526.7**	35.1	56.6	1733.6	179.3
Ours	799.9	69.2	**1234.7**	785.6	**17.2**	**21.6**	**813.6**	**8.6**

nents of GMM_P and GMM_T when $k \neq \ell$. Therefore, we have

$$\Pr[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)] = \sum_{k=1}^{K_P} \pi_{P_k}\pi_{T_k} P_{intra}^{(k)} + \sum_{k\neq\ell} \pi_{P_k}\pi_{T_\ell} P_{\text{cross}}^{(k,\ell)} \xrightarrow{\lambda\to\infty} \sum_{k=1}^{K_P} \pi_{P_k}\pi_{T_k} P_{intra}^{(k)}, \tag{45}$$

Finally, under the circumstances, we achieve

$$\Pr[s_i = \mathrm{NN}_{\mathcal{T}}(P_i)] \geq 1 - \sum_{k=1}^{K_P} \pi_{P_k} \cdot \exp\left(-\beta(d,k) \cdot \frac{\pi_{T_k}}{\pi_{P_k}} \cdot \frac{N}{K}\right) \tag{46}$$

□

C. Hyperparameter for training

Table VI summarizes the training hyperparameters used across all datasets. To accommodate datasets of varying characteristics and sizes, we employed different batch sizes (ranging from 4 to 64) and training epochs (from 50 to 200). Larger datasets were trained with bigger batch sizes for computational efficiency, while smaller datasets utilized smaller batches with more epochs to ensure adequate learning. This tailored approach ensures stable convergence and optimal performance for each dataset.

D. Running Time Comparison with ROCKET

To better demonstrate the efficiency improvement of TSketches, we additionally compare the running time of TSketch + vanilla Transformer (Ours) with ROCKET, as shown in Table VII. The running time is reported in seconds.

The results show that ROCKET is faster on the relatively shorter datasets (RWC and KLC), while our method becomes faster on the other six longer datasets. This is because transformer models can better benefit from the parallel processing capability of GPUs, whereas ROCKET is typically executed on CPU. This observation is also consistent with the findings from a few recent papers [45], [46], which shared an author with ROCKET.

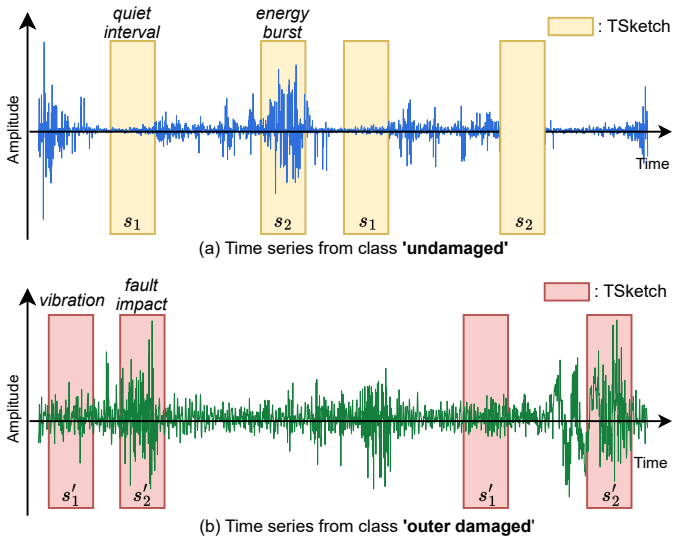


Fig. 19. TSketches of two time series from dataset FaultDetectionA.

E. Case Study on FaultDetectionA

We further present a case study on the FaultDetectionA dataset. As shown in Figure 19, we randomly sample one time series from each of the 'undamaged' and 'outer damaged' classes, and visualize the TSketches s_1 and s_2 for the former, and s'_1 and s'_2 for the latter.

First, our method discovers TSketches that naturally correspond to meaningful operating states in bearing vibration signals. In the undamaged sample, the discovered sketches s_1 and s_2 correspond to a *quiet interval* and an *energy burst*, respectively. This pattern is consistent with the normal operation of an electromechanical drive system, where the bearing produces periodic vibration responses while remaining relatively stable between these responses. Consequently, the original time series can be effectively represented as an alternating sequence of *quiet intervals* and *energy bursts*.

Meanwhile, the TSketches of the damaged sample show a marked divergence from the undamaged one. Specifically, s'_1 corresponds to an *inter-impact vibration*, while s'_2 corresponds to a *fault impact*. The *inter-impact vibration* represents the relatively ordinary vibration between such impact events, whereas the *fault impact* reflects the short-term high-amplitude vibration caused when rolling elements pass over a damaged region of the bearing. Compared with the undamaged sample, whose local structure is dominated by regular *quiet intervals* and *energy bursts*, the damaged sample exhibits more abrupt and irregular impact patterns.